raffle.fun

WHITEPAPER · VERSION 2.0

raffle.fun

A Plain-English Whitepaper for Onchain NFT Raffles

How prize escrow, ticket ownership, randomness, payouts, refunds, and recovery work

Target Base (chain ID 8453) Commit f165e4c1 Date August 10, 2026 Status Internal review only

Heesho

ABOUT THIS DOCUMENT

Document and review record

FIELD	VALUE
Title	raffle.fun: A Plain-English Whitepaper for Onchain NFT Raffles
Author	Heesho
Document version	2.0
Protocol version reviewed	raffle.fun contracts 2.0.0 (monorepo 0.1.0)
Reviewed repository commit	`f165e4c1d8f5d093fe0a36094f79a29857c26286`
Reviewed contract commit	`f165e4c1d8f5d093fe0a36094f79a29857c26286`
Internal-audit source identity	`a2120f5e163dc3641d9864773febbfedca047edb` plus patch fingerprint `55589ce1fce0bd5579e60df747796575df9b0d96`; the final reviewed implementation was committed as `f165e4c1d8f5d093fe0a36094f79a29857c26286`
Production Solidity after audit baseline	Yes. The constructor/bearer refactor and internal-review remediations changed production Solidity after the baseline SHA. Those changes are included in the internally reviewed final commit above. No independent audit covers them.
Publication date	August 10, 2026
Target network	Base (chain ID 8453)
Deployment status	Not deployed and not authorized for user funds
Testnet deployment status	No verified Base Sepolia deployment record
Internal-review status	Security review status: Internal adversarial review only
Independent-audit status	Independent external audit: Not completed
Legal-review status	Legal review: Not completed
Compiler / dependencies	Solidity 0.8.36 · OpenZeppelin 5.6.1 · Pyth Entropy SDK 2.2.1
License	Repository license applies. Embedded Nunito and Inter fonts use the SIL Open Font License 1.1.

DISCLAIMER

This whitepaper explains how the raffle.fun smart contracts work at one reviewed source-code commit. It is an explanatory document, not an offer, a promise of future performance, or an inducement to participate. The protocol distributes a prize by chance. Gambling, lottery, sweepstakes, contest, consumer-protection, sanctions, tax, licensing, and age-restriction laws may apply differently across jurisdictions. Nothing here is legal, tax, investment, or financial advice. Anyone who operates or uses the protocol must obtain advice for their own jurisdiction. Smart-contract testing and internal review reduce risk but cannot eliminate it, and this document describes unaudited software: do not commit assets you cannot afford to lose unless and until the protocol has received appropriate independent review, legal review, and deployment validation. Raffle tickets are entries in a prize drawing. They are not investments, they carry no expectation of profit, and most tickets win nothing.

CONTENTS

What is in this whitepaper

How to Read This Paper

This paper explains the raffle.fun contracts committed at `f165e4c1d8f5d093fe0a36094f79a29857c26286`. It serves two readers at once. The main text is for an intelligent reader who has never used a blockchain. The callouts, formulas, state tables, and appendices provide the precision needed by sponsors, developers, auditors, operators, and legal reviewers.

Read Parts I through VI for the product and its economic outcomes. Read Parts VII through X for refunds, claims, architecture, and administration. Read Part XI before placing value at risk. Parts XII and XIII are practical guides and questions. The appendices are the contract-level reference.

IN ONE SENTENCE

raffle.fun escrows one NFT, sells transferable ERC-721 bearer tickets for one factory-wide USDC token, requests one Pyth Entropy random value, and then lets the rightful bearers or fixed recipients redeem the resulting assets.

Three reading rules

1. **Solidity wins.** If this paper, a website, an indexer, or an earlier design note disagrees with production Solidity, the contract controls onchain behavior.
2. **A recorded right is not the same as a completed transfer.** A settlement can create a quote claim or identify a winning ticket before anyone receives the asset.
3. **Every safety statement has assumptions.** The supported recovery claim assumes honest token contracts, available transfers, a functioning Base chain, and a user able to submit transactions.

The continuous example

The paper follows a fictional raffle for **Pixel Passport #42**, a fictional one-of-one NFT. Sofia is the sponsor. Her secure recovery wallet is the fixed sponsor-side prize-recovery recipient. Tickets cost 10.00 USDC. The minimum threshold is 100 tickets. The sale lasts seven days. Maya, Leo, Noor, and other fictional buyers hold tickets. Alex is an unrelated keeper who may request the draw.

All names, assets, addresses, and activity are illustrative. The economic values are generated by the build from compiled Solidity constants. They are not claims about actual users, volume, partners, or deployment.

A compact vocabulary

Blockchain. A shared transaction record maintained by a network. Think of it as a public ledger whose accepted entries are replayed by many computers.

Base. An Ethereum layer-2 network. It executes Ethereum-compatible smart contracts and posts data to Ethereum, while its sequencer orders Base transactions.

Wallet. Software and keys used to sign a transaction. A wallet does not store an NFT inside the app; the blockchain records which address owns it.

Transaction. A signed request to run a contract function. Transactions may fail, may be delayed, and require gas.

Gas. The execution fee paid in the chain's native currency. USDC ticket payments and native-currency gas or Entropy fees are different assets.

Smart contract. Program code and persistent state executed by the blockchain.

NFT and ERC-721. An NFT is a distinct token. ERC-721 is the standard interface the protocol uses for both the prize and each raffle ticket.

ERC-20 and quote token. ERC-20 is a fungible-token interface. The quote token is the factory-wide USDC contract used to price tickets and pay cash outcomes.

Escrow. Custody under fixed rules. During a raffle, the raffle contract owns the configured prize NFT.

Threshold. The minimum sold-ticket count that makes successful randomness award the NFT. It is not a sales cap.

Oracle and entropy. An oracle brings externally produced data onchain. Pyth Entropy v2 produces the random value used by raffle.fun.

Random callback. A later transaction from the configured Entropy contract that delivers the random value for one accepted sequence.

Bearer ticket. The current owner of an unburned ticket owns the related winning or refund right. The ticket must be burned to consume that right.

Pull claim. A recorded balance that the claimant withdraws in a later transaction. Sponsor and treasury proceeds use pull claims. Ticket awards and refunds use bearer redemptions.

Factory. The contract that validates creation, deploys raffles, registers them, and deposits each prize atomically.

Indexer and subgraph. Offchain software that reconstructs convenient history from events. It can be stale or wrong and does not control settlement.

Immutable. Not changeable through the contract's available functions. Each raffle is independently deployed and has no upgrade path.

Permissionless. Any address may call the function if its onchain preconditions are met. It does not mean every parameter or external dependency is unrestricted.

Multisig. A contract wallet that may require several approved signers. A final factory-owner multisig has not been selected or activated in a verified deployment.

Reentrancy. A called contract attempts to call back before the first operation finishes. raffle.fun uses state ordering and reentrancy guards around asset flows.

Basis points. Hundredths of a percentage point. 500 basis points equals 5%.

DOCUMENT STATUS

Deployment status: Not deployed and not authorized for user funds. Security review status: Internal adversarial review only. Independent external audit: Not completed. Legal review: Not completed.

PART I

The Idea

raffle.fun replaces manually administered custody, ticket records, random selection, and payout bookkeeping with a fixed onchain process.

PART I

The Idea

1. Executive Summary

raffle.fun is experimental software for one-prize NFT raffles on Base. A sponsor chooses the price, the minimum-ticket threshold, the sale dates, and a fixed recovery recipient. The canonical factory deploys one independent raffle, registers it, moves the exact prize NFT into escrow, and verifies custody in one transaction. If any step fails, every step rolls back.

The factory is bound to one quote-token contract. The reviewed implementation calls this token USDC and deployment tooling requires the official chain-specific USDC address. Users therefore do not select an arbitrary ERC-20 when creating a raffle. The factory also fixes one Pyth Entropy v2 address and one callback gas limit.

Any user may create a raffle while new creation is not paused, using the factory-wide USDC and an ERC-721 prize contract that reports ERC-721 interface support. This is open creation with bounded inputs, not completely unrestricted creation.

A buyer approves USDC and calls `buyTickets`. The complete gross price is `ticketPrice x quantity`. The raffle checks that its balance increased by exactly that amount before it safely mints sequential ERC-721 ticket IDs. A purchase may mint from 1 through 100 tickets. Total sales can continue above the minimum threshold until the exclusive sale end.

Every ticket is a bearer right. It remains transferable in every lifecycle state until it is burned. There is no ownership snapshot and no draw-time transfer freeze. If Maya transfers ticket 12 to Leo, Leo owns the potential award or refund. If Leo then transfers it to Noor before redemption, Noor becomes the owner of that right. Only the actual current owner, not merely an approved operator, can burn a winning or refundable ticket.

After the sale, anyone may pay the current Pyth Entropy fee and request the one draw. The request is permitted at `endTime` and strictly before `endTime + 3 days`. The fee is paid in Base's native currency, not from the USDC ticket pot. Any overpayment is returned immediately. If the return fails, the complete request transaction reverts.

The later Entropy callback is accepted only through the configured Entropy wrapper, for the stored sequence, while the raffle is `Drawing`, and after the request call has finished. Wrong, early, stale, duplicate, and post-failure callbacks are ignored. A valid callback computes:

```
winningTicketId = (randomNumber % totalTickets) + 1
```

The mapping includes every ticket ID from 1 through `totalTickets`. It has negligible but nonzero mathematical modulo bias for most ticket counts. Pyth correctness remains an external assumption.

If the threshold is met, status becomes `NftWon`. The winning ticket owner burns the ticket for the NFT. The protocol treasury receives a 5% pull claim and the sponsor receives the complete post-fee USDC pot as a separate pull claim.

If the threshold is missed but a valid callback arrives, status becomes `CashWon`. This is a successful cash fallback, not a refund. The winning ticket owner burns the ticket for 80% of the post-fee pot. The sponsor receives the exact remainder. The protocol receives the same 5% fee. The fixed recovery recipient claims the NFT. Other ticket owners receive no refund.

If no request succeeds by the request-grace deadline, or no valid callback wins before timeout finalization at `drawRequestedAt + 2 days`, anyone may call `enableRefunds`. No winner is selected, no fee is charged, and no sponsor proceeds are created. Each current owner may burn up to 100 owned tickets in one transaction for exactly one ticket price per ticket. Tickets remain transferable until burned.

If zero tickets sell, the sponsor may close the raffle before the end and anyone may close it at or after the end. The recovery recipient then claims the NFT. There is no separate cancellation status. `Closed` is the zero-sales outcome.

Sponsor and treasury quote proceeds are pull claims so one recipient cannot block another. Winning cash and refunds are direct exact transfers made when bearer tickets burn. The recovery recipient initiates its NFT claim and chooses a safe nonzero destination. `claimQuoteFor` may be called by anyone, but can pay only the rightful account

itself.

The factory owner can pause future creation, change the treasury captured by future raffles, and use OpenZeppelin's two-step ownership transfer. The owner cannot change, pause, upgrade, settle, rescue, or redirect an existing raffle.

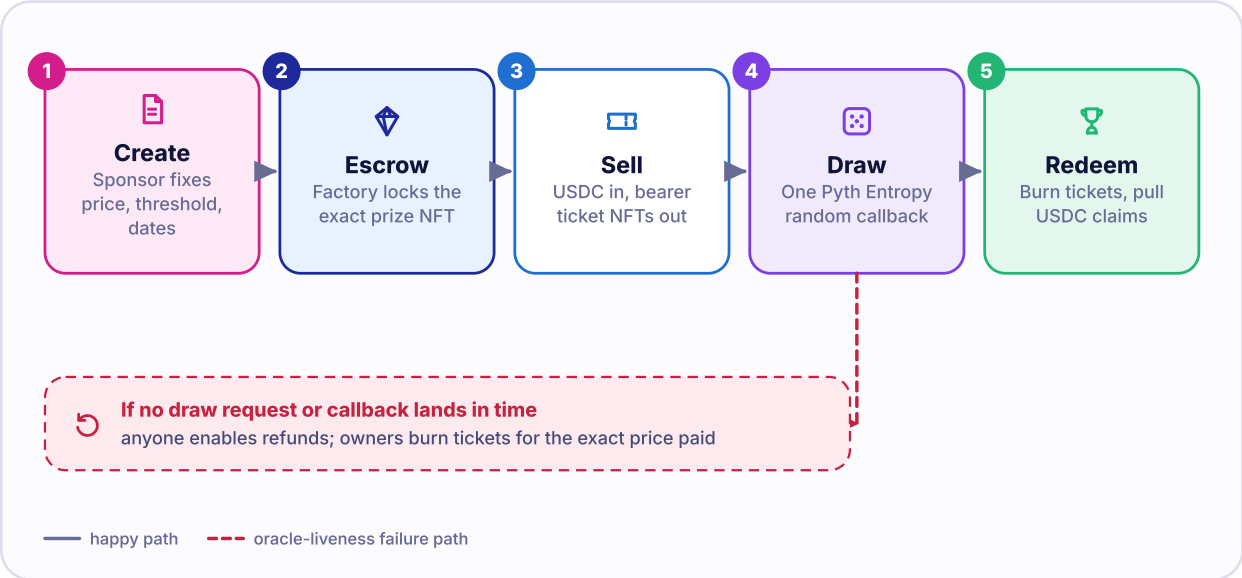
The largest risks are external token behavior, oracle correctness, Base ordering and availability, wallet security, user-selected non-callable ticket destinations, immutable-code defects, and legal or operational failures. Internal testing is strong evidence, not proof that vulnerabilities do not exist.

WHAT THIS DESIGN OPTIMIZES FOR

The design favors fixed rules, independent asset claims, bounded recovery deadlines, and minimal administrator power over upgradeability or discretionary rescue.

2. raffle.fun in 60 Seconds

FIGURE 1 **raffle.fun at a glance**



A successful raffle moves from atomic prize escrow through ticket sales and one randomness request to bearer redemption; oracle-liveness failure moves instead to exact bearer refunds.

1. **Create.** Sofia selects Pixel Passport #42, 10.00 USDC tickets, a 100-ticket threshold, seven sale days, and her recovery wallet.
2. **Escrow.** The factory deploys and registers one raffle, then transfers and verifies the exact NFT in the same transaction.
3. **Sell.** Buyers pay the full ticket price and receive transferable ticket NFTs.
4. **Request.** After the sale, Alex or anyone else pays Pyth's current native fee.
5. **Resolve.** A valid callback selects one ticket. The threshold decides NFT versus cash, not whether a winner exists.
6. **Recover from liveness failure.** Missing request or callback deadlines enable exact ticket refunds without an administrator.
7. **Redeem.** Current ticket owners burn winning or refundable tickets. Sponsor and treasury withdraw recorded quote claims. The recovery recipient claims the NFT in cash, refund, or empty outcomes.

3. The Problem

A manually operated raffle asks participants to trust several facts that may be hard to verify. Who owns the prize during the sale? Can the sponsor change the terms? Were all tickets counted? Who produced the random result? Who can move the money? What happens if the random service never answers?

The problem is not that every operator is dishonest. The problem is that informal processes often leave custody, timing, and failure recovery ambiguous. A spreadsheet can record ticket numbers but cannot prevent an administrator from editing the sheet. A livestream can show a random draw but may not prove the prize was continuously held or the winning number was mapped correctly.

Common weaknesses include:

- unclear prize custody;
- mutable sale terms;
- discretionary winner selection;
- opaque payment handling;
- no defined oracle-failure recovery;
- payouts that depend on one operator acting later;
- dependence on one website to explain or execute the result.

raffle.fun addresses only the onchain mechanics. It does not establish whether a particular raffle is lawful, whether metadata describes a genuine asset, whether a token issuer will keep transfers available, or whether participants understand the risk.

4. The Core Idea

The contract is like a transparent lockbox with a rulebook attached. Sofia places the prize in the lockbox. Buyers receive numbered bearer receipts. After the deadline, an external randomness service supplies a value that the lockbox maps to one receipt. The lockbox then exposes only the asset paths belonging to the selected outcome.

The rulebook moves seven responsibilities into contracts:

- **custody:** the raffle owns the configured NFT;
- **ticket ownership:** ERC-721 ownership identifies current bearer rights;
- **timing:** inclusive start, exclusive end, request grace, and callback timeout;
- **randomness authentication:** only the configured Pyth Entropy wrapper can deliver the stored sequence;
- **payout accounting:** successful branches allocate every raw USDC unit;
- **refunds:** failure branches reserve the gross pot for exact ticket burns;
- **claims:** independent recipients can retry their own asset transfers.

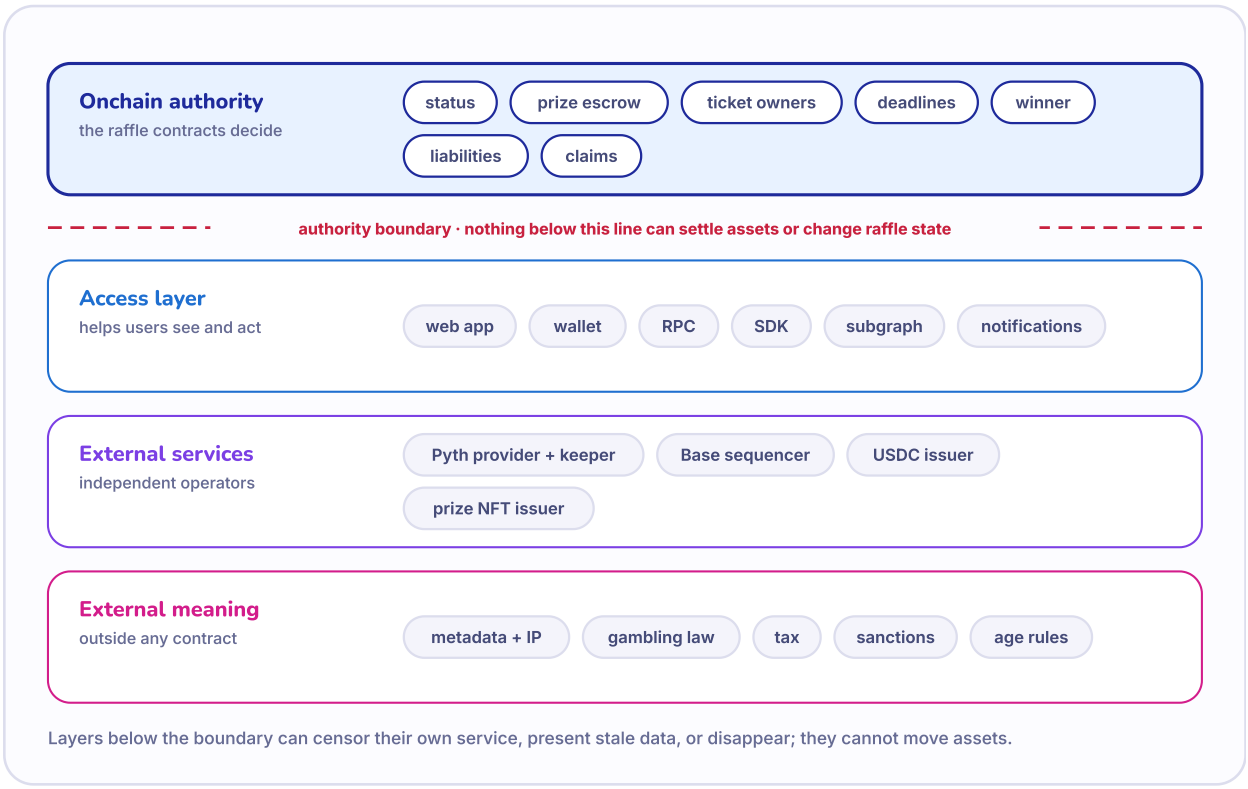
> **WHAT THE CONTRACT CANNOT GUARANTEE**
> The contract cannot force a frozen USDC token to transfer, make a malicious NFT tell the truth, restore lost wallet keys, compel a Base transaction to be included, or make a chance-based promotion legal in every jurisdiction.

5. What Onchain Means

Onchain does not mean every part of the experience is automatic or free from external dependencies. It means the authoritative rules and state are available through the contract on Base.

ONCHAIN AUTHORITY	OFFCHAIN OR EXTERNAL DEPENDENCY
Raffle configuration and registered address	Website presentation and transaction prompts
Prize and ticket ownership	RPC availability and indexing latency
Start, end, grace, and timeout timestamps	Wallet software and key custody
Accepted Entropy sequence and raffle status	Pyth provider and keeper infrastructure
Winning ticket ID and quote liabilities	NFT metadata hosting and truthfulness
Bearer burns, claims, and emitted events	USDC issuer controls and prize-contract behavior
Factory owner and future-only policy	Legal classification, sanctions, tax, licensing, and age rules

FIGURE 2 **Onchain versus offchain**



The blockchain is authoritative for raffle state and asset ownership. Access layers, metadata, external services, and legal meaning remain outside the raffle contracts.

If the frontend disappears, a technically capable user can still read the contracts and submit transactions through another interface. That does not remove dependency on Base, an RPC endpoint, a wallet, or the external token contracts.

PART II

People and Assets

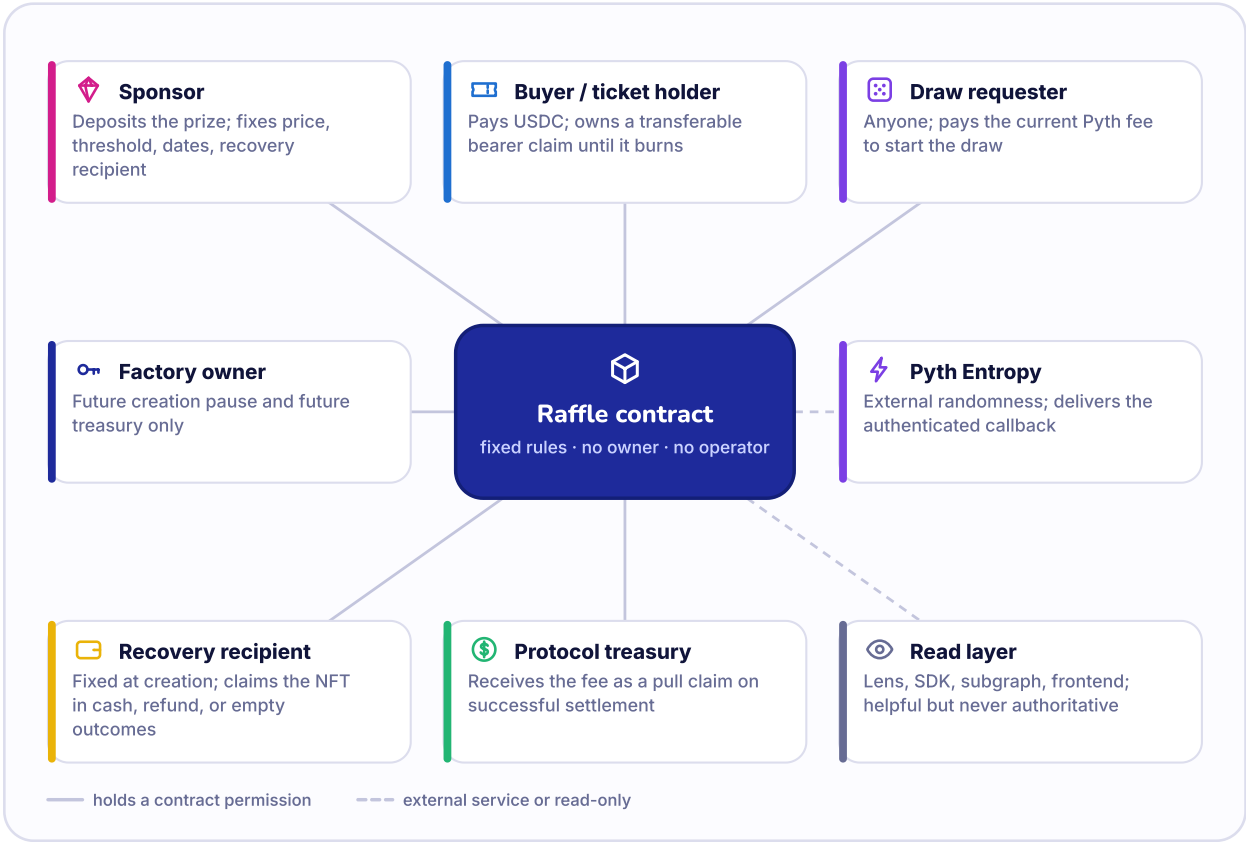
The protocol separates prize custody, bearer ownership, fixed recovery rights, randomness delivery, and future-only administration.

PART II

People and Assets

6. Participant Roles

FIGURE 3 **Participant role map**



One address may hold several roles, but each contract permission is defined separately.

Sponsor. The account that calls the factory and deposits the prize. It selects the prize, price, threshold, dates, metadata, and optional recovery recipient. It receives normal sponsor USDC proceeds. It does not own or administer the deployed raffle.

Sponsor-side prize-recovery recipient. The fixed address that may withdraw the NFT after `CashWon`, `Refunding`, or `Closed`. A zero creation input defaults to the sponsor. The recipient may choose a safe nonzero destination when it claims. It cannot be changed later.

Buyer. The account that pays USDC. The buyer may mint tickets to another recipient, so payment and ticket ownership can belong to different accounts.

Ticket recipient and current holder. The recipient receives new ticket NFTs. After any transfers, the current owner holds the bearer right. Only the current owner may burn a winning or refundable ticket, even if another account has transfer approval.

Winner. The owner of the selected unburned ticket when redemption occurs. Winner identity can change after selection because the ticket remains transferable until it burns.

Draw requester. Any account that pays at least the current Entropy fee during the request window. The requester need not be a sponsor or ticket holder and receives no reimbursement from the USDC pot.

Refund finalizer. Any account that calls `enableRefunds` after the applicable liveness deadline. The finalizer does not receive the refund money and does not select owners.

Quote claimant. The sponsor or protocol treasury when successful settlement creates its pull claim. A claimant can choose a nonzero destination with `claimQuote`.

Prize claimant. The winning bearer in `NftWon`, or the fixed recovery recipient in `CashWon`, `Refunding`, and `Closed`.

Protocol treasury. The account captured by a raffle at creation. It receives the successful-settlement fee as a quote pull claim. Later factory treasury updates do not change it.

Factory owner. The OpenZeppelin `Ownable2Step` administrator for future creation pause and future treasury selection. No existing raffle grants this owner a selector.

Pyth Entropy. The configured external randomness contract. It accepts the request and later authenticates callback entry. Oracle correctness and infrastructure remain external assumptions.

Frontend, SDK, Lens, and subgraph. Convenience layers. The Lens authenticates registered raffle addresses and reads bounded state. The SDK simulates writes. The subgraph indexes events. None can choose the winner or settle a raffle.

ROLE	CAN DO	CANNOT DO	MUST TRUST OR VERIFY
Sponsor	Create terms; deposit prize; claim successful proceeds	Cancel after a sale; change deployed terms; seize prize	Factory address, token behavior, recovery address, legality
Recovery recipient	Claim NFT in cash, refund, or empty result	Change itself; take NFT from `NftWon`	Key security and destination capability
Buyer	Pay USDC; choose ticket recipient	Force settlement; obtain a general cash-fallback refund	Price, dates, quote token, recipient, transaction result
Current ticket holder	Transfer ticket; burn winner or refunds	Redeem without actual ownership; redeem twice	Ticket ID, status, destination, Base inclusion
Draw requester	Pay current Pyth fee and request once	Choose random value; charge the ticket pot	Fee quote, request window, overpayment return capability
Refund finalizer	Enable deadline-based refunds	Select refund owner or take a fee	Correct deadline and chain inclusion
Treasury	Claim successful fee	Change existing fee or divert other claims	Factory creation snapshot and token availability
Factory owner	Pause future creation; change future treasury; transfer ownership	Upgrade, pause, rescue, reroll, or settle an existing raffle	Owner-key or multisig operations
Pyth Entropy	Deliver callback through authenticated wrapper	Override status after terminal transition	Provider, keeper, and oracle correctness
Frontend or subgraph	Present and index data	Authoritatively settle or change contract state	Correct deployment configuration and fresh reads

7. The Prize NFT

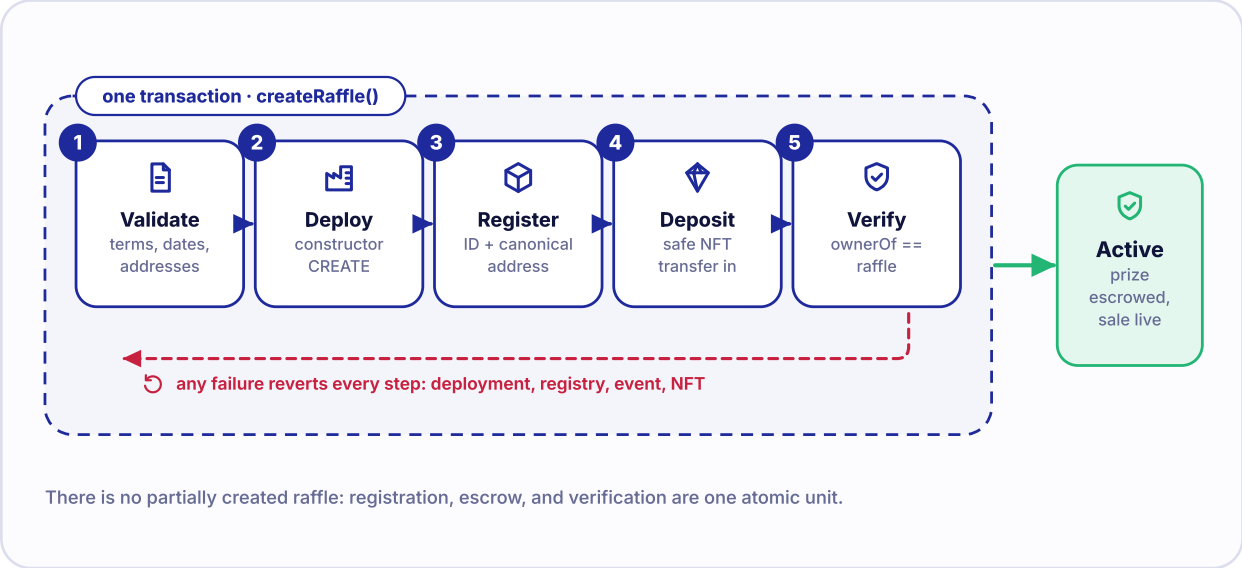
An ERC-721 token represents one distinct token ID in one collection contract. Pixel Passport #42 is identified by both its collection address and token ID 42. A picture or name displayed by a website is metadata, not the ownership record.

Creation requires the prize address to contain code and report support for the ERC-721 interface through ERC-165. The factory then performs a safe transfer from the sponsor to the new raffle. The raffle receiver accepts only:

- the configured prize contract;
- the configured token ID;
- the immutable sponsor as `from`;
- the canonical factory as `operator`;
- the `AwaitingPrize` status.

After the transfer, the factory checks that `ownerOf(prizeTokenId)` equals the raffle and that the raffle entered `Active`. Failure reverts the complete creation.

FIGURE 4 **Atomic creation and prize escrow**



Deployment, registry assignment, exact prize deposit, and verification succeed together or roll back together.

This verification proves only what the prize contract reports in that transaction. It cannot make malicious or upgradeable NFT code honest. A prize contract could later lie about ownership, burn the escrowed token, pause transfers, reject transfers, reenter, or change metadata.

Unsafe ERC-721 `transferFrom` can force unrelated NFTs into a raffle without calling the receiver hook. Such tokens are outside protocol accounting. There is no factory- owner rescue function and no generic raffle sweep.

IMPORTANT PRIZE RISK

Use only a prize whose contract, upgrade controls, ownership reporting, safe-transfer behavior, and metadata provenance have been independently evaluated. Interface support alone is not a safety certification.

8. The Quote Token

The quote token is the fungible asset used for ticket payments and cash outcomes. Every raffle from one factory uses the same immutable quote-token address. In the reviewed implementation and deployment validation, that asset is chain-specific USDC. Sponsors do not choose from an admission registry.

USDC commonly uses six decimal places. A human display of 10.00 USDC corresponds to 10,000,000 raw units. Solidity performs all accounting in raw integers.

Before minting tickets, the raffle measures its quote-token balance, calls `transferFrom`, measures again, and requires the increase to equal the gross price. Before paying USDC, it measures both the raffle and destination balances and requires the exact expected debit and exact expected credit. A tax, fee, surcharge, dishonest balance, or unexpected rebase makes the operation revert.

These checks reject many unsupported behaviors but do not guarantee permanent availability. USDC may be paused, blacklisted, upgraded, or otherwise controlled by its issuer. A later issuer action can trap an otherwise valid claim.

Direct USDC donations are not ticket purchases. They do not increase `totalTickets`, `grossSales`, the fee, sponsor proceeds, winner cash, or refunds. They appear only as surplus above accounted liabilities. No administrator can sweep that surplus.

9. Ticket NFTs

Each ticket is an ERC-721 token minted by its raffle. Ticket IDs begin at 1 and remain sequential. One ticket equals one chance in the winner formula. A bounded purchase can mint several IDs to one recipient.

The ticket is a bearer credential:

- while unburned, ownership can transfer in `Active`, `Drawing`, `NftWon`, `CashWon`, `Refunding`, or `Closed`;
- a selected winning ticket carries its NFT or cash right to each new owner;
- a refundable ticket carries its one-ticket-price refund right to each new owner;
- burning the ticket consumes that right and the ERC-721 token ceases to exist.

Known protocol destinations are rejected because they may own a ticket but lack a way to initiate redemption. A ticket cannot transfer to its own raffle, its factory, the quote token, the Entropy contract, the prize contract, or a registered sibling raffle.

This cannot identify every incapable third-party contract. An owner can use unsafe transfer to send a ticket to an unrelated contract that cannot call redemption. That ticket may become unrecoverable. A narrow helper covers future same-factory raffle addresses that were code-less at transfer time; it is not a generic rescue.

All tickets may share one metadata URI. Metadata can be unavailable or misleading. Tickets are receipts and bearer ownership records, not investments and not guaranteed valuable collectibles.

PART III

Creating a Raffle

Creation fixes every raffle-specific asset, party, price, threshold, timestamp, and dependency before the prize enters escrow.

PART III

Creating a Raffle

10. Creation Inputs

Sofia calls `RaffleFactory.createRaffle` with eight sponsor-controlled fields:

INPUT	PLAIN MEANING	PIXEL PASSPORT EXAMPLE
`prizeToken`	ERC-721 collection contract	fictional Pixel Passport collection
`prizeTokenId`	exact prize within that collection	42
`sponsorPrizeRecoveryRecipient`	fixed NFT recipient for cash, refund, or empty outcome	Sofia's secure recovery wallet
`ticketPrice`	complete raw USDC price for one ticket	10,000,000 raw units = 10.00 USDC
`minimumTickets`	sold count selecting NFT rather than cash after a valid callback	100
`startTime`	inclusive sale start; zero means the creation timestamp	immediate
`endTime`	exclusive sale end	seven days after start
`metadataURI`	shared raffle and ticket metadata URI	fictional HTTPS or content-addressed URI

The sponsor does not supply the quote token, Entropy address, callback gas limit, protocol treasury, factory address, or raffle ID. The factory fixes or assigns those values.

The displayed ticket price is the full gross price. It is not a deposit, pre-fee amount, or partial contribution. Economic fees are allocated only after a successful random callback.

11. Creation Bounds

The factory rejects a creation unless:

- the prize address has deployed code and reports ERC-721 support;
- `ticketPrice` is nonzero;
- `minimumTickets` is nonzero;
- a nonzero scheduled start is not in the past;
- start is no more than 7 days after creation;
- end is strictly after the normalized start;
- sale duration is no more than 30 days;
- the metadata URI is at most 2048 bytes;
- future creation is not paused.

These are custody and execution bounds. They do not decide whether the chosen price, threshold, duration, prize, or promotional structure is commercially sensible or legally permitted.

The minimum threshold is not a supply cap. A threshold of 100 means that a valid draw with 100 or more sold tickets awards the NFT. It does not stop ticket 101, 120, or 10,000 from being sold before the sale ends.

> **FOR SPONSORS**
> Verify the recovery recipient character by character. It cannot be changed after creation. A zero input intentionally defaults to the sponsor, but a separate secured wallet may reduce operational risk if it can call `claimSponsorPrize`.

12. Atomic Escrow

The factory uses ordinary constructor `CREATE`. It does not deploy an EIP-1167 clone, an initializer proxy, a CREATE2 address, or an upgradeable raffle. Each new raffle has its own runtime code, storage, tickets, liabilities, and assets.

Creation follows one transaction:

1. validate the sponsor-controlled fields;
2. normalize a zero start to the current block timestamp;
3. assign the next raffle ID;
4. constructor-deploy an independent `Raffle` with all fixed values;
5. register its address in both ID mappings and `isRaffle`;
6. emit the indexer-complete `RaffleCreated` event;
7. safe-transfer the configured NFT from the sponsor to the raffle;
8. have the raffle receiver validate token, ID, sponsor, operator, and status;
9. verify `ownerOf` and `Active` after transfer.

An Ethereum transaction is atomic: either the complete sequence succeeds or every state and asset movement reverts. A rejecting prize, dishonest postcondition, reentrant factory attempt, wrong approval, or receiver mismatch cannot leave a successfully registered but unfunded raffle.

The event is emitted before the external NFT transfer in execution order, but a later failure reverts the event with the transaction. Indexers should not treat a reverted log as creation evidence.

> **UNDER THE HOOD**
> The raffle constructor begins at `AwaitingPrize` and accepts only its declared factory as `msg.sender`. The safe receiver moves it to `Active`. No successfully completed factory creation should leave a registered raffle at `AwaitingPrize`.

13. The Recovery Recipient

The recovery recipient exists because sponsor identity and operational asset custody need not be the same address. Sofia can sponsor the raffle from one account while her security-controlled wallet is responsible for recovering Pixel Passport #42 if the NFT is not awarded.

The recovery recipient receives the right to claim the NFT in three result groups:

- `CashWon`, after a valid draw below the threshold;
- `Refunding`, after the request or callback liveness deadline;
- `Closed`, after zero sales.

It does not receive sponsor USDC proceeds merely because it holds this NFT right. Successful sponsor quote claims belong to the immutable sponsor address. It cannot claim the NFT in `NftWon`, where the winning bearer holds the prize right.

The recipient chooses a safe nonzero destination at claim time. If the destination rejects the NFT, the transaction reverts and `prizeClaimed` returns to its prior value, so the authorized recipient can retry.

Known protocol addresses cannot be configured as unsafe fixed recipients. The constructor rejects itself, the factory, quote token, Entropy contract, prize token, and registered sibling raffles. A permissionless selective helper covers a narrower case where a future same-factory raffle address was selected while still code-less. Recovered assets route only to the holding raffle's immutable recovery recipient.

PART IV

Ticket Sales

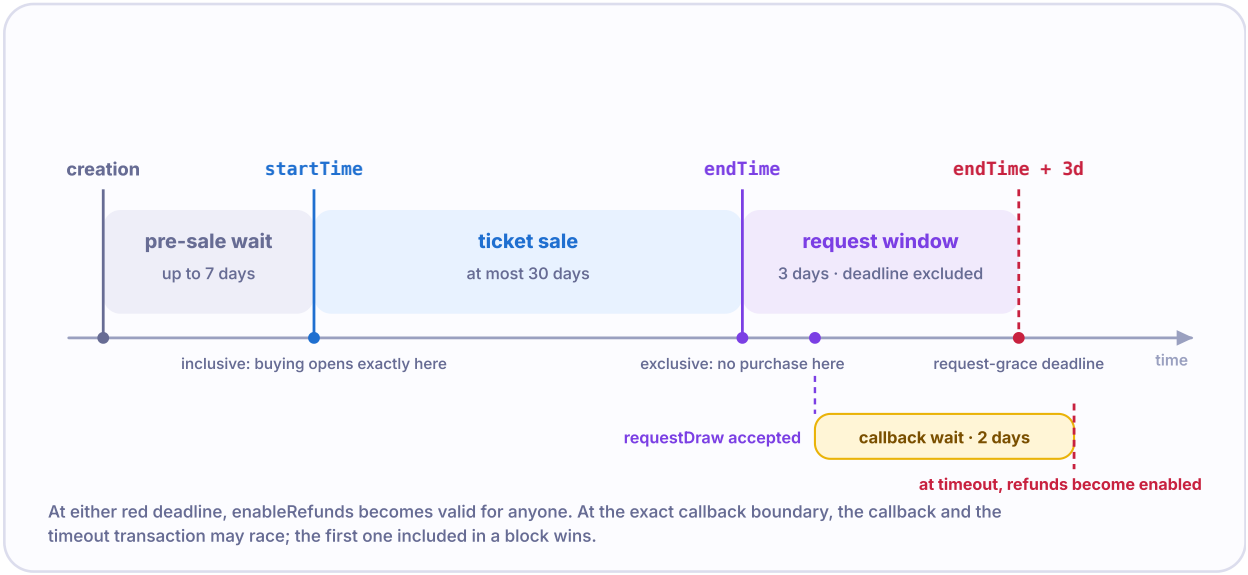
Buyers pay the exact gross USDC price and receive sequential, transferable ERC-721 bearer tickets.

PART IV

Ticket Sales

14. Sale Timeline

FIGURE 5 **Sale and deadline timeline**



The inclusive sale start and exclusive sale end are followed by a strict request window and an exact callback-timeout boundary.

Purchases are allowed when all three conditions hold:

`status == Active`

`block.timestamp >= startTime`

`block.timestamp < endTime`

The exact start second is included. The exact end second is excluded. At `endTime`, a sold raffle may accept a draw request and a zero-sales raffle may be closed by anyone.

Block timestamps and transaction ordering are supplied by the chain. A wallet sending near a boundary cannot guarantee inclusion before the boundary. Users should avoid last-second assumptions.

15. Buying Tickets

A buyer usually completes two USDC transactions. First, the buyer approves the raffle to spend enough USDC. Second, the buyer calls `buyTickets(recipient, quantity)`.

The raffle requires:

- `Active` status and an open sale window;
- a nonzero recipient;
- a quantity from 1 through 100;
- safe multiplication of price by quantity;
- an exact USDC balance increase equal to the gross cost.

Only after exact payment does the raffle update `grossSales` and `unsettledPot`, assign the next ticket range, and safely mint each ERC-721 to the recipient. A rejecting contract recipient makes the complete purchase revert, including USDC payment and all tentative ticket mints.

If Maya buys three tickets when 11 already exist, she receives IDs 12, 13, and 14. The purchase event records the payer, recipient, quantity, first and last IDs, and gross raw amount.

> **FOR TICKET BUYERS**
> Check the raffle address, chain, ticket price, quantity, recipient, USDC allowance, and expected gross amount in the wallet. A successful approval does not itself buy a ticket.

16. Threshold Behavior

The threshold selects the economic branch only after a valid random callback:

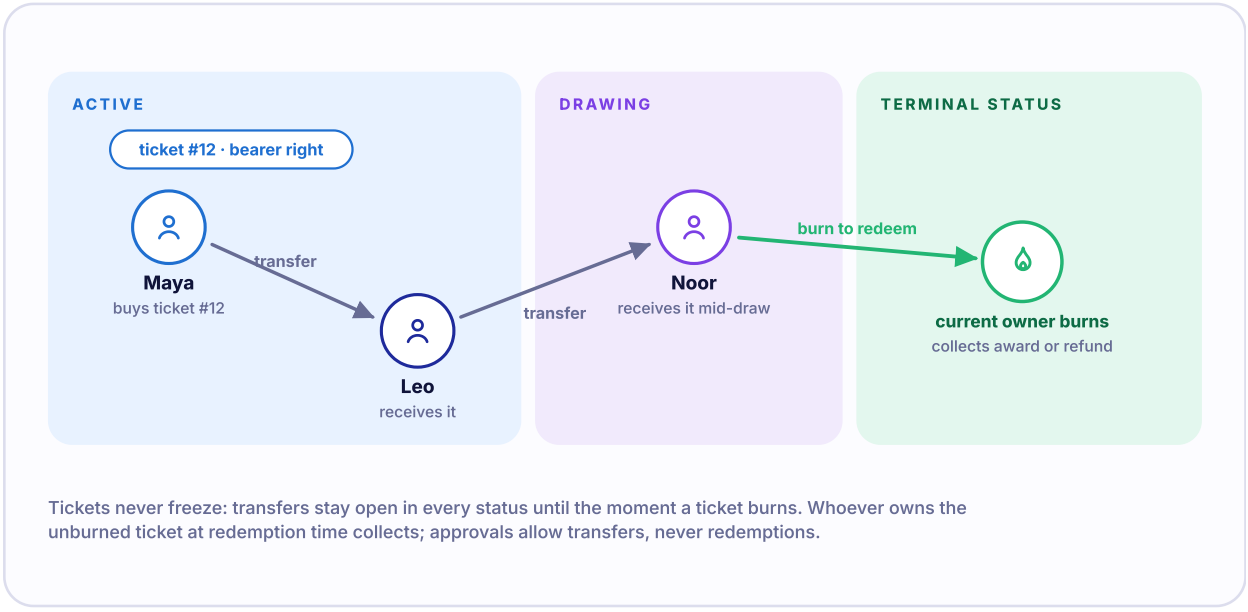
SOLD TICKETS	100-TICKET THRESHOLD	SUCCESSFUL RESULT
99	below	`CashWon`
100	equal	`NftWon`
101	above	`NftWon`

Equality counts as meeting the threshold because Solidity tests `totalTickets >= minimumTickets`. Sales may continue above 100 until `endTime`.

The threshold does not change ticket price, odds, request timing, oracle source, fee rate, or failure refunds. It only decides whether successful randomness gives the winning bearer the NFT or cash.

17. Ticket Ownership and Transfers

FIGURE 6 **Bearer ownership through time**



There is no transfer freeze or ownership snapshot. An unburned ticket carries its right to each current owner until redemption consumes it.

Maya buys ticket 12 and transfers it to Leo during the sale. If ticket 12 is selected, Leo may transfer it to Noor even after the callback. Noor then becomes the actual owner who may burn it for the NFT or cash. If refunds are enabled instead, the current owner may transfer ticket 12 before someone burns it for 10.00 USDC.

This is materially different from a snapshot design:

- draw request does not freeze transfers;
- refund finalization does not snapshot refund owners;
- no refund is pre-credited to a past owner;
- a transferred unburned ticket moves the unconsumed right;
- after burn, no souvenir ticket remains.

ERC-721 approval lets another account transfer a ticket, but the raffle redemption functions require `msg.sender` to equal `ownerOf(ticketId)`. An approved operator cannot directly redeem without first becoming the owner.

Safe and unsafe ERC-721 transfers both pass through raffle.fun's overridden `transferFrom` logic for known protocol-destination checks. Safe transfer additionally asks a contract recipient to confirm ERC-721 receipt. Unsafe transfer to an arbitrary incapable contract can still lock the bearer right.

18. Odds

For one selected winning ticket, a holder's simple share of the ticket set is:

`tickets currently owned / total tickets sold`

If Maya owns 4 of 80 tickets, her current ownership share is 5%. That share changes when more tickets sell or tickets transfer. The contract selects a ticket ID, not a wallet. One wallet may own several IDs and one ID has one position in the mapping.

The formula describes ticket share, not a guarantee of winning. Pyth supplies a 256-bit value that is reduced with modulo. For most ticket counts, the 256-bit domain is not divisible exactly by `totalTickets`, creating negligible but nonzero modulo bias. The paper therefore does not claim perfect mathematical uniformity.

Cash fallback does not give every ticket a partial payout. It still selects one ticket. Only its current owner may burn for the winner cash amount.

PART V

Randomness and Deadlines

One paid Pyth Entropy v2 request can resolve the raffle; two fixed deadlines bound oracle-liveness failure.

PART V

Randomness and Deadlines

19. Why an Oracle Is Needed

Smart-contract execution is deterministic. Every validating computer must reach the same result from the same transaction. A contract cannot privately roll dice.

raffle.fun does not let the sponsor, factory owner, frontend, or requester supply the winning number. It does not use block timestamp, block hash, or `prevrandao` alone. Those chain values can be visible or influenceable around inclusion and are not the reviewed oracle design.

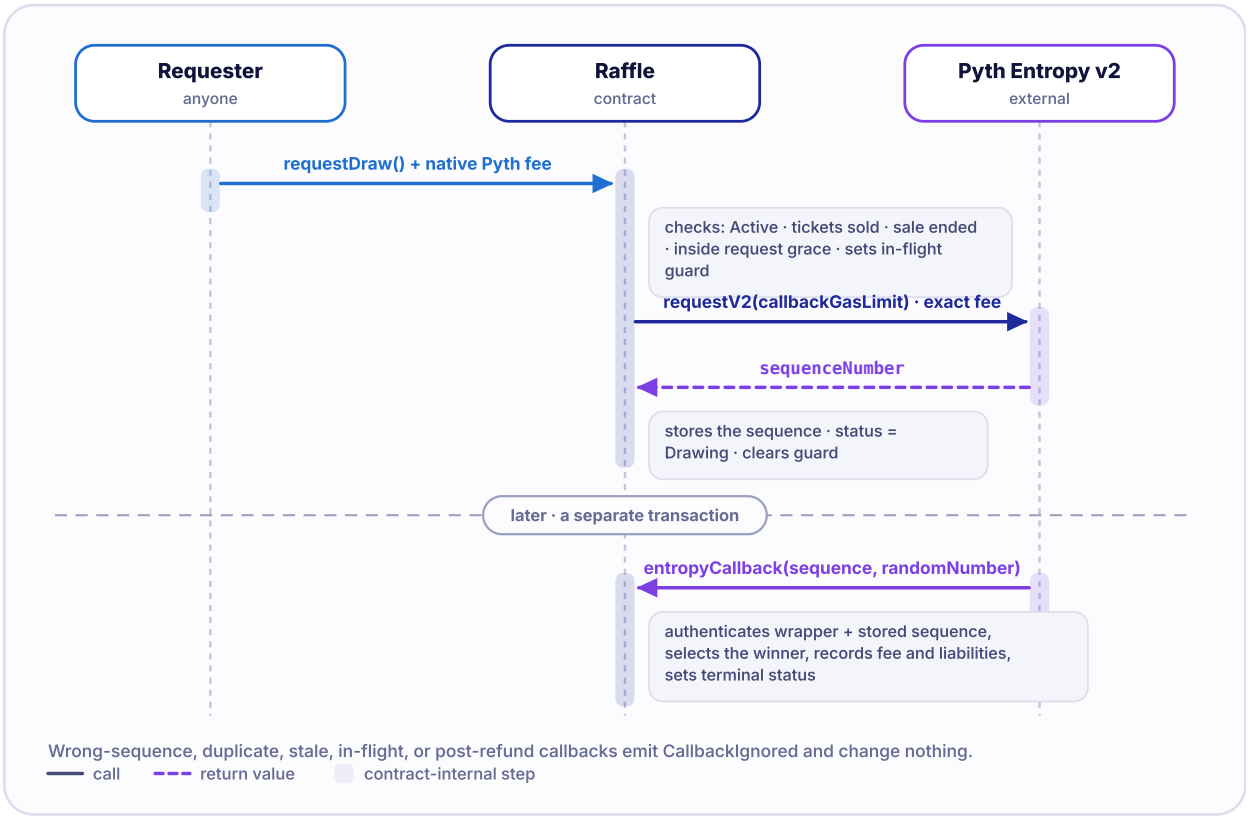
Pyth Entropy v2 separates the request from the later random callback. That separation creates a liveness problem, so raffle.fun also defines what happens if the request or callback never succeeds.

20. Pyth Entropy v2

Pyth Entropy is an external random-number service for EVM contracts. The raffle reads the current fee for its immutable callback gas limit, pays that fee, and receives a sequence number. A provider or keeper later submits a separate callback transaction through the configured Entropy contract.

The protocol authenticates the wrapper and sequence, but oracle correctness is still an external assumption. The default Entropy model, provider operation, fee policy, keeper availability, and official deployment address must be independently verified at deployment time.

FIGURE 7 **Randomness request and callback**



The request and callback are separate Base transactions. The callback records only bounded settlement state and performs no token transfer.

21. Requesting the Draw

`requestDraw` is payable and permissionless. It requires:

- `Active` status;

- at least one sold ticket;
- the sale to have ended;
- the current timestamp to be strictly before the request-grace deadline;
- native value at least equal to `getFeeV2(callbackGasLimit)`.

Before calling Entropy, the raffle enters `Drawing`, records `drawRequestedAt`, and sets a private in-flight guard. It calls `requestV2(callbackGasLimit)` with exactly the quoted fee, stores the returned sequence, and clears the guard.

If Alex supplies more native value than required, the raffle immediately calls Alex with the exact excess. It does not keep a native pull claim. If Alex's account or contract rejects the return, the entire request rolls back to `Active`; the raffle does not consume its one request or retain the payment.

The USDC ticket pot never reimburses the requester. Request gas and Pyth's native fee are separate operating costs.

22. Callback Authentication

The Pyth SDK exposes an external wrapper that authenticates the calling Entropy contract before entering raffle.fun's internal callback. The raffle then accepts settlement only if:

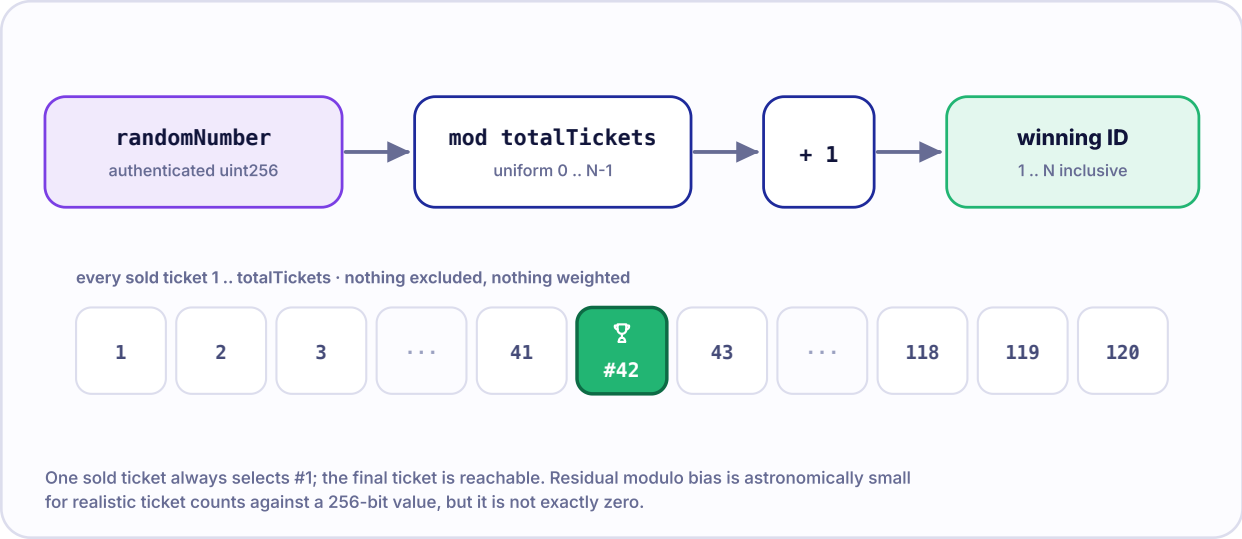
- the request is no longer in flight;
- status is exactly `Drawing`;
- the received sequence equals the stored sequence.

An in-flight, wrong-sequence, stale, duplicate, or post-refund callback emits `EntropyCallbackIgnored` and returns without settlement. Ignoring instead of reverting helps keep oracle delivery behavior observable without letting an obsolete callback rewrite a terminal result.

The valid callback performs bounded storage work. It selects the ticket, calculates fee and payout amounts, records quote liabilities, clears `unsettledPot`, and enters `NftWon` or `CashWon`. It calls no USDC contract, prize contract, user, sponsor, or treasury.

23. Selecting the Winner

FIGURE 8 **Winner selection**



Modulo maps the authenticated 256-bit value into the complete one-based sold-ticket range.

The formula is:

`(uint256(randomNumber) % totalTickets) + 1`

Modulo first returns a number from 0 through `totalTickets - 1`. Adding one maps it to ticket IDs 1 through `totalTickets`.

- ticket 1 is reachable;
- the final sold ticket is reachable;
- a one-ticket raffle always selects ticket 1;
- no unsold ID is reachable;
- the formula is deterministic once the random value and sold count are known.

Modulo bias is negligible but nonzero unless the 256-bit input domain divides evenly by the sold count. The contract does not attempt rejection sampling or a second oracle request.

24. The Request Grace Period

The request-grace deadline is:

`endTime + 3 days`

At `endTime`, request is allowed. At the exact grace deadline, request is no longer allowed and `enableRefunds` becomes allowed if tickets exist. This strict boundary prevents a missing request from holding the prize and pot indefinitely.

Fee-read failure, changing fee, insufficient payment, Entropy request revert, or failed native overpayment return leaves the raffle `Active`. Until the deadline, another account may try again. At the deadline, failure finalization wins instead.

25. The Callback Timeout

The callback deadline is:

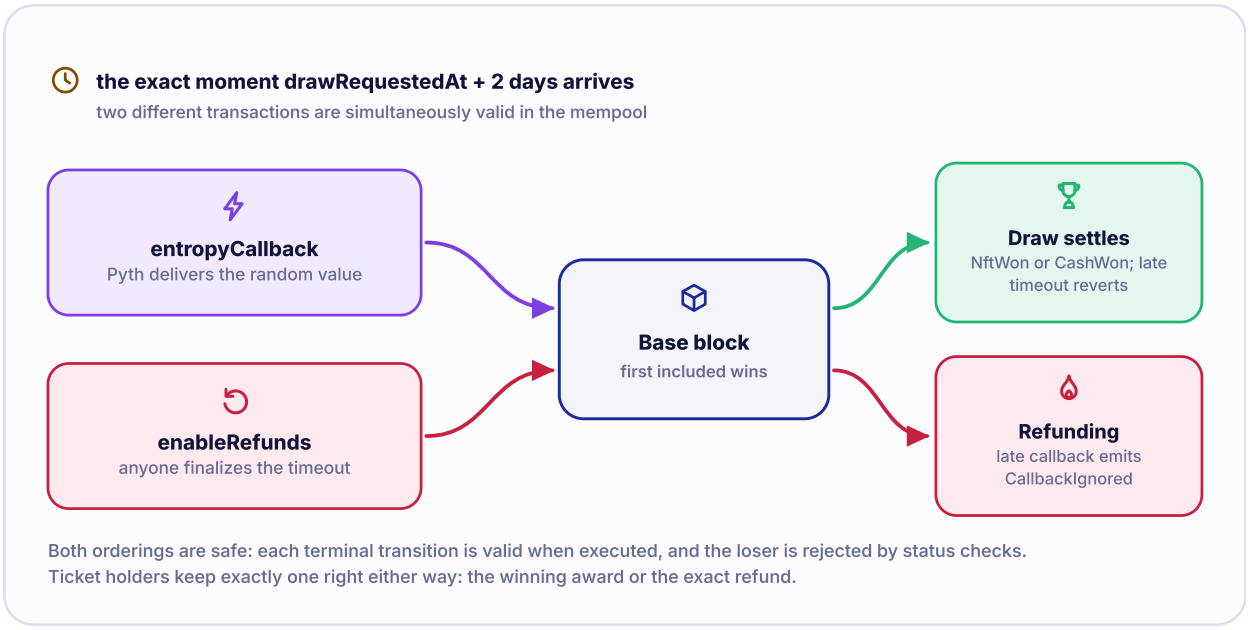
`drawRequestedAt + 2 days`

Before the deadline, `enableRefunds` reverts from `Drawing`. At the deadline, anyone may call it. The function moves the entire `unsettledPot` into `remainingRefundLiability`, enters `Refunding`, and emits whether a request had been accepted.

The nominal deadline does not make a callback invalid by timestamp alone. The callback checks status and sequence, not time. If no timeout transaction has been included yet, a valid callback may still settle at or after the nominal deadline.

26. The Boundary Race

FIGURE 9 **Callback versus timeout**



At the exact callback deadline, both transactions may satisfy their own checks before inclusion. Base ordering determines which valid terminal transition executes first.

At the callback deadline, two transactions can be valid against the same pre-state:

- Pyth's matching callback can enter `NftWon` or `CashWon`;
- any account's `enableRefunds` can enter `Refunding`.

The first transaction included against `Drawing` determines the result. If the callback wins, timeout finalization later sees a terminal successful status and reverts. If timeout wins, the later callback is ignored because status is no longer `Drawing`.

This is deterministic given transaction inclusion and order, but it does not remove sequencer ordering, delay, censorship, or reorganization risk.

ORACLE LIVENESS IS BOUNDED, ORACLE TRUST IS NOT REMOVED

Deadlines prevent indefinite waiting under normal chain inclusion. They do not prove the random value correct, guarantee a callback, guarantee a refund transaction will be included, or defeat a halted or universally censored chain.

PART VI

Economic Outcomes

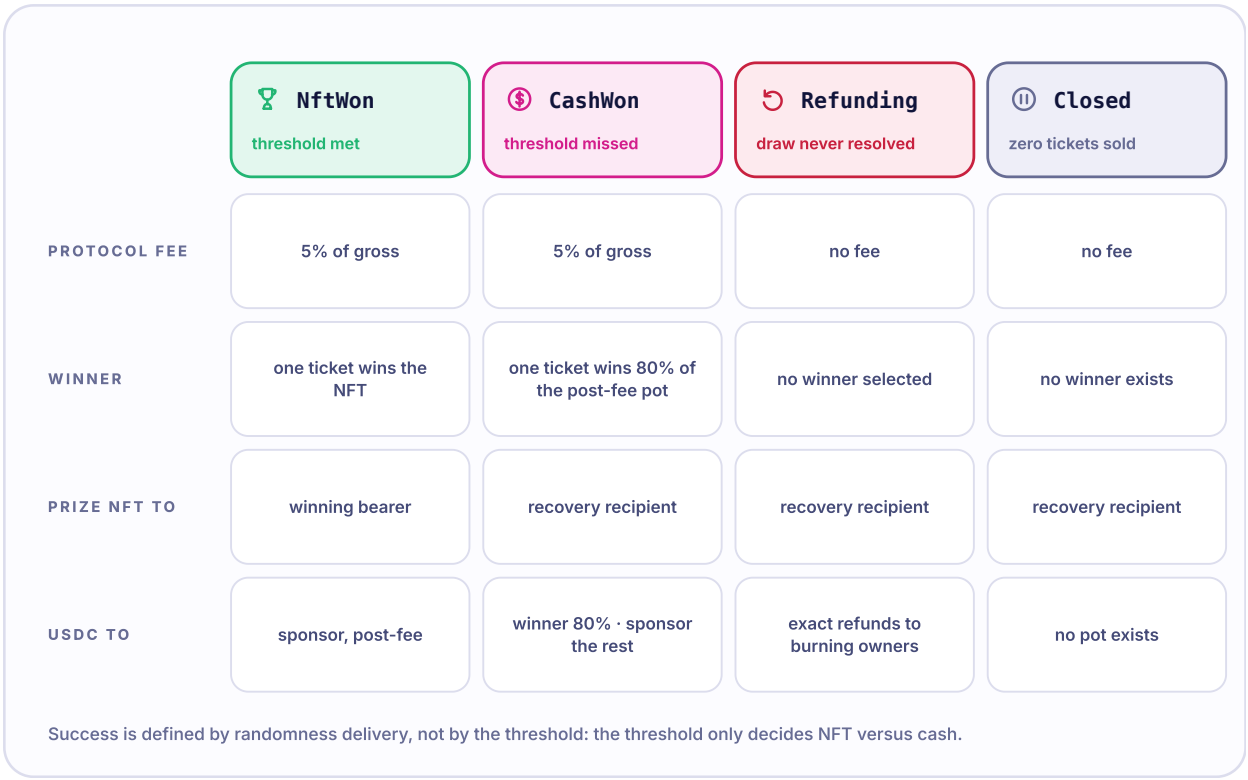
Successful randomness charges the same protocol fee in NFT and cash branches. Oracle-liveness failure charges no fee and reserves the gross pot for exact ticket refunds.

PART VI

Economic Outcomes

27. Outcome Overview

FIGURE 10 **Outcome comparison**



Successful randomness produces one selected ticket and a fee. Liveness failure produces no winner or fee. Zero sales closes without a request.

STATUS	TRIGGER	WINNING BEARER	PROTOCOL FEE	SPONSOR USDC	NFT CLAIMANT	TICKET REFUNDS
`NftWon`	valid callback; sold count at least threshold	burns selected ticket for NFT	5% of gross, floored	all post-fee USDC	winning bearer	none
`CashWon`	valid callback; sold count below threshold	burns selected ticket for 80% of post-fee pot	5% of gross, floored	post-fee arithmetic remainder	fixed recovery recipient	none for other tickets
`Refunding`	missing request or callback deadline	none	none	none	fixed recovery recipient	each current bearer burns for exact price

STATUS	TRIGGER	WINNING BEARER	PROTOCOL FEE	SPONSOR USDC	NFT CLAIMANT	TICKET REFUNDS
	finalized					
`Closed`	zero sold tickets	none	none	none	fixed recovery recipient	none because none sold

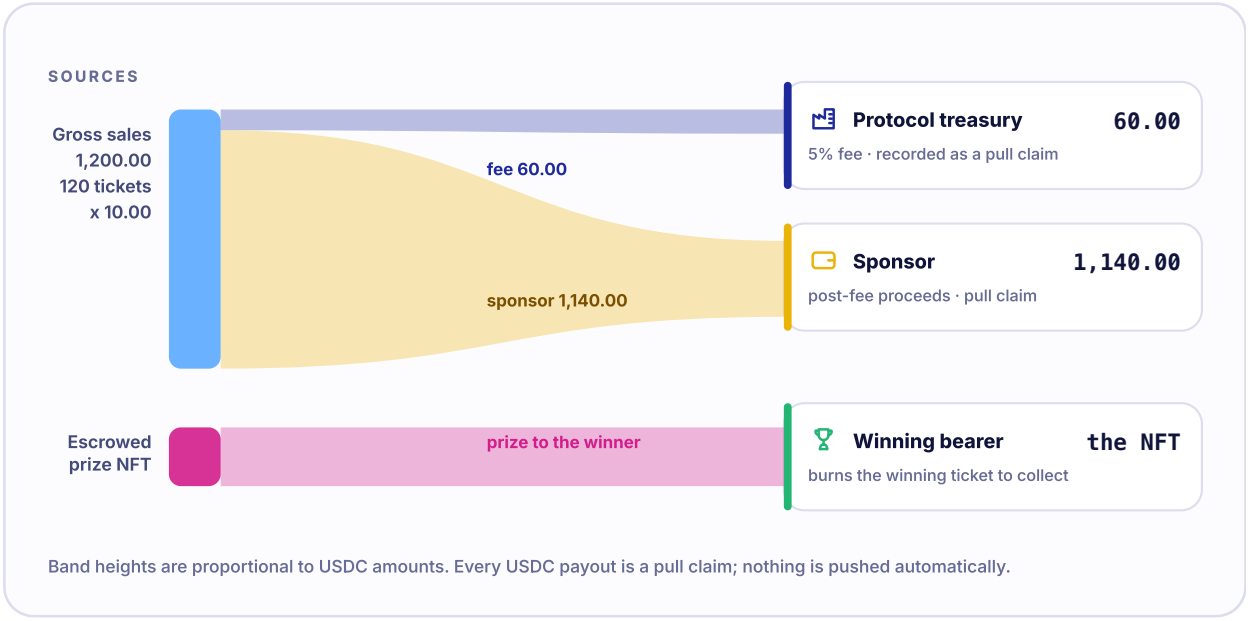
`AwaitingPrize`, `Active`, and `Drawing` are nonterminal lifecycle statuses. The winning and recovery claims can remain unredeemed after a terminal status. A status records the economic result, not proof that every external transfer has completed.

28. NFT Awarded

Scenario A sells 120 tickets at 10.00 USDC with a threshold of 100:

ALLOCATION	HUMAN AMOUNT	RAW SIX-DECIMAL UNITS
Gross sales	1,200.00 USDC	`1200000000`
Protocol fee	60.00 USDC	`60000000`
Sponsor claim	1,140.00 USDC	`1140000000`
Winner	Pixel Passport #42	selected ticket burns

FIGURE 11 **NFT-awarded money flow**



The selected bearer burns for the NFT. Sponsor and treasury USDC are independent pull claims.

The callback credits the treasury and sponsor; it does not transfer USDC. The selected ticket remains transferable until an owner calls `redeemWinningTicket(to)`. The burn occurs before safe transfer of the prize. If the destination rejects the NFT, the complete transaction reverts and restores the ticket and prize state.

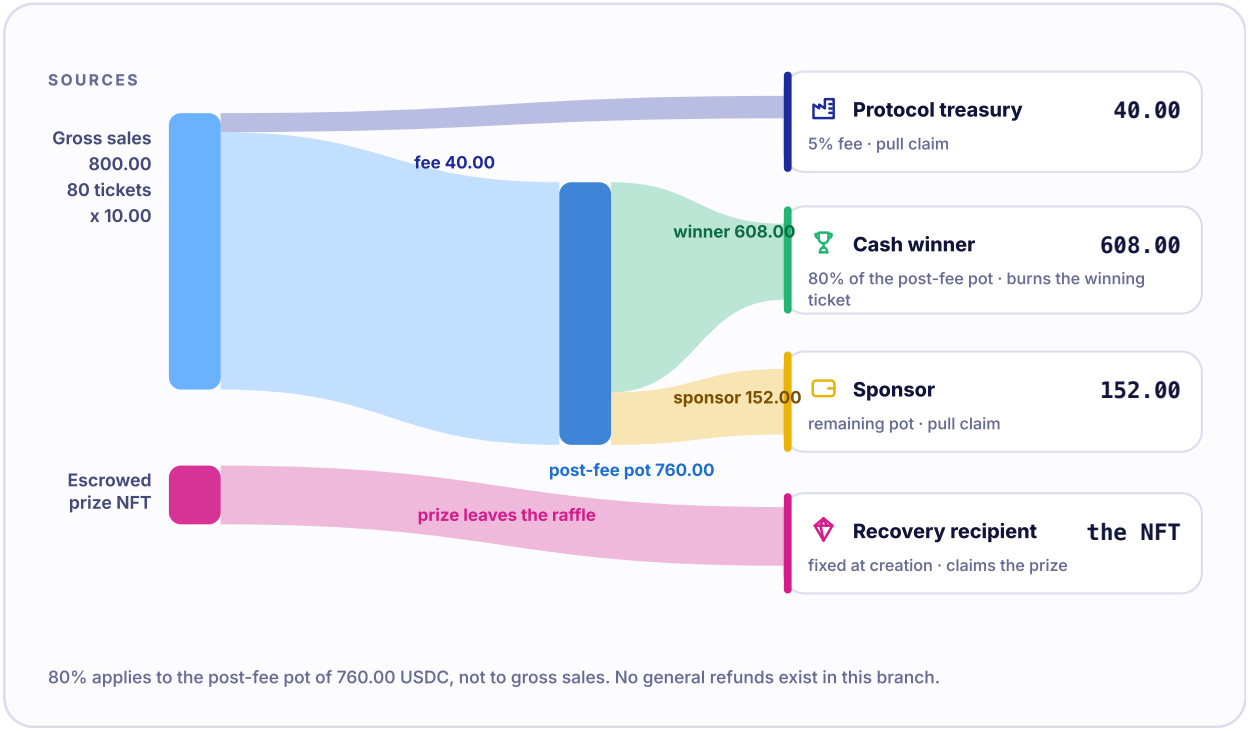
The sponsor can claim 1,140.00 USDC to a safe nonzero destination. The treasury can claim 60.00 USDC independently. Failure by either recipient does not block the winner or the other quote claimant.

29. Cash Fallback

Scenario B sells 80 tickets, below the threshold, and receives a valid callback:

ALLOCATION	HUMAN AMOUNT	RAW SIX-DECIMAL UNITS
Gross sales	800.00 USDC	`800000000`
Protocol fee	40.00 USDC	`40000000`
Post-fee distributable	760.00 USDC	gross minus fee
Winning bearer cash	608.00 USDC	`608000000`
Sponsor cash remainder	152.00 USDC	`152000000`
Recovery recipient	Pixel Passport #42	separate NFT claim

FIGURE 12 **Cash-fallback money flow**



The winner share is 80% of the post-fee pot, not of gross sales. The recovery recipient claims the NFT.

The callback records the winner cash as `winnerCashLiability` and sponsor and treasury amounts as ordinary quote claims. The selected ticket owner burns the ticket and receives the exact cash amount in the same transaction. The recovery recipient claims the NFT separately.

Cash fallback is a successful raffle outcome. It does not refund all buyers. Other ticket holders receive no payment merely because the threshold was missed.

30. No Sales

Scenario E ends with zero tickets. No Entropy request is needed and there is no USDC liability. At or after `endTime`, anyone may call `closeEmptyRaffle`. Status becomes `Closed`, and the fixed recovery recipient may claim the prize.

Before `endTime`, the sponsor may call the same function if no tickets have sold. This lets the sponsor end an unused raffle without waiting. Once ticket 1 is sold, `closeEmptyRaffle` is permanently unavailable because `totalTickets` never decreases, even when tickets later burn.

31. Early Empty Close

The current contract does not expose a general `cancel` function or `Cancelled` status. The sponsor's only early-close path is a zero-sales close. This matters for public wording:

- before any sale, the sponsor may close and the recovery recipient may claim the NFT;
- after any sale, the sponsor cannot cancel, change terms, or reclaim the NFT by discretion;

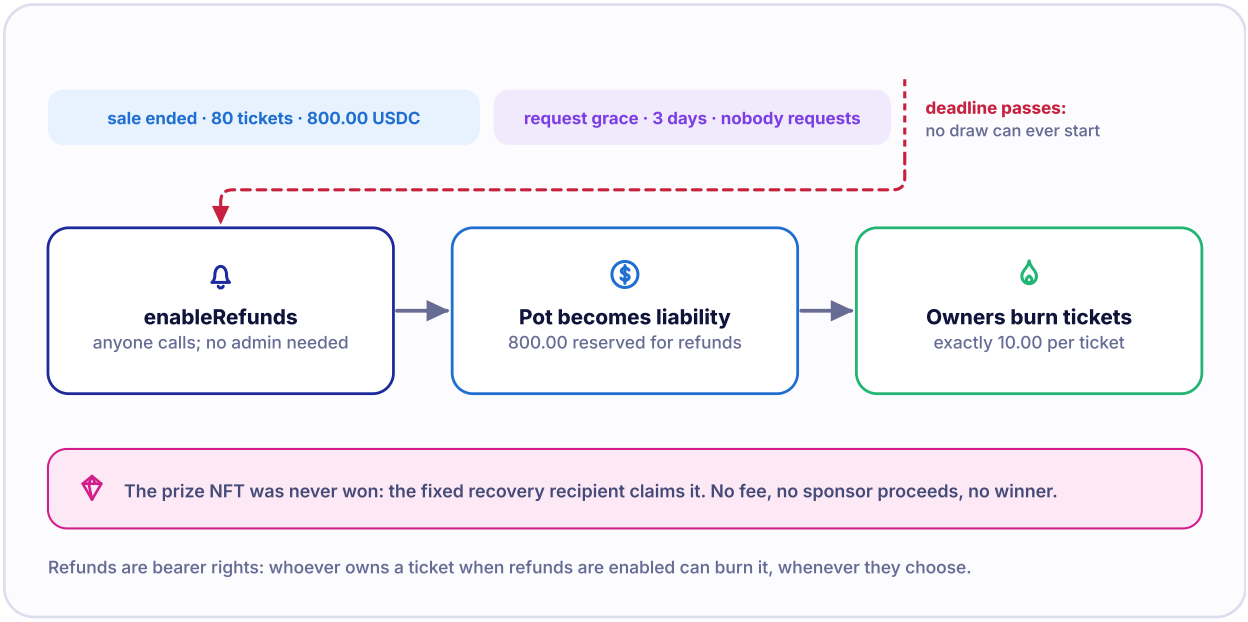
- a sold raffle must follow draw, cash/NFT settlement, or deadline refunds.

This is the accurate counterpart to the common phrase "cancellation before sales." It should not be described as an administrator override.

32. Draw Not Requested

Scenario C uses the same 80-ticket, 800.00 USDC gross pot, but no draw request succeeds before the grace deadline.

FIGURE 13 **Missing-request refund flow**



At the request-grace deadline, anyone may enable exact bearer refunds. There is no winner, fee, or sponsor proceeds.

When anyone calls `enableRefunds` at or after the deadline:

- status moves from `Active` to `Refunding`;
- `unsettledPot` becomes zero;
- `remainingRefundLiability` becomes 800.00 USDC;
- no winning ticket is selected;
- no treasury fee or sponsor claim is created;
- the recovery recipient can claim Pixel Passport #42.

Each current ticket owner may burn owned tickets for 10.00 USDC each. All 80 tickets together account for `800000000` raw units.

33. Draw Timed Out

Scenario D accepts a valid request and freezes nothing. Ticket transfers remain available. If no matching callback wins before timeout finalization, anyone calls `enableRefunds` at or after `drawRequestedAt + 2 days`.

The financial outcome is the same as missing-request failure: no winner, no fee, no sponsor proceeds, exact bearer refunds, and the NFT for the recovery recipient. The event records that a request had been accepted.

A late callback after `Refunding` emits an ignored callback event and cannot reverse refund liability. At the exact timeout boundary, transaction order decides whether the callback or refund finalization wins.

34. Fee and Rounding Math

The successful-settlement formulas are:

`grossSales = ticketPrice x totalTickets`

`protocolFee = floor(grossSales x 500 / 10,000)`

`distributablePot = grossSales − protocolFee`

For `NftWon`:

`sponsorCash = distributablePot`

For `CashWon`:

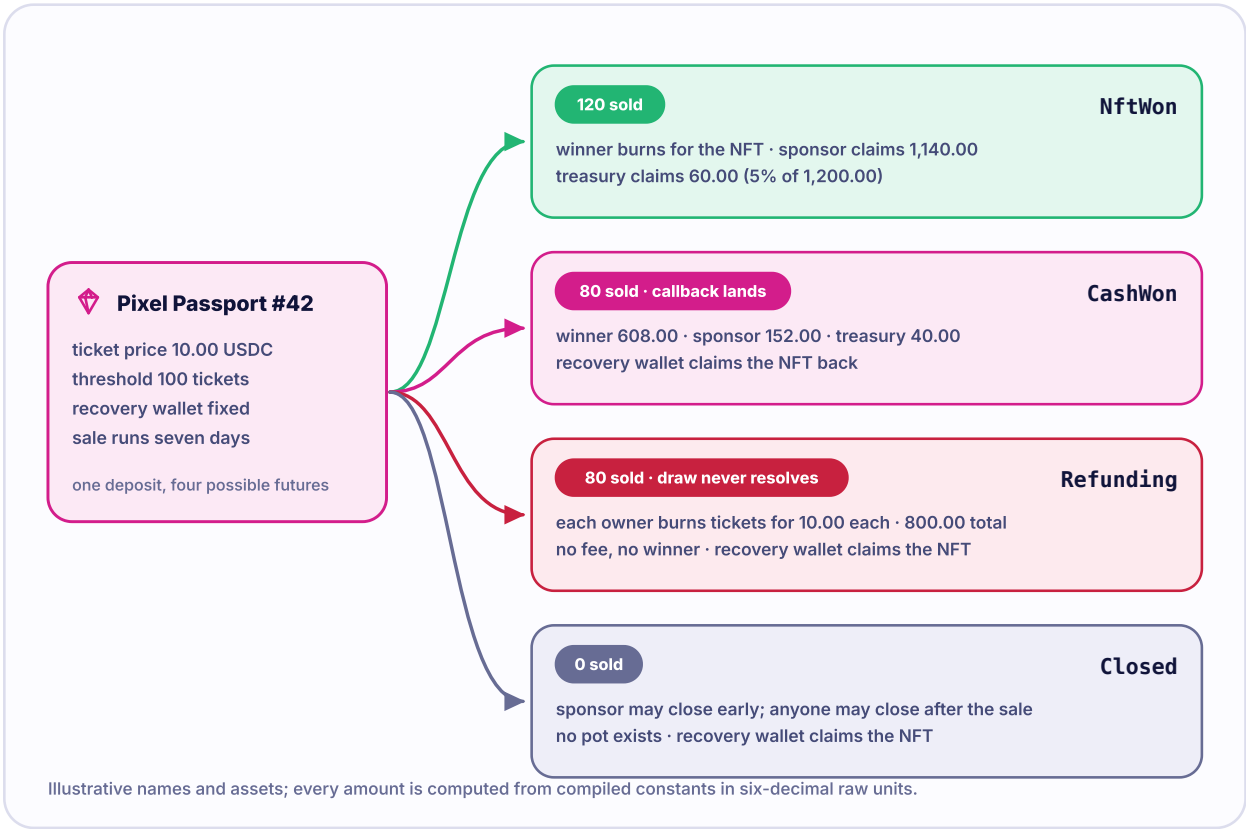
`winnerCash = floor(distributablePot x 8000 / 10,000)`

`sponsorCash = distributablePot − winnerCash`

Solidity uses integer arithmetic. `Math.mulDiv` performs multiplication and division with floor rounding. The sponsor receives the arithmetic remainder after winner cash, so the fee, winner amount, and sponsor amount allocate every raw unit.

The winner gets 80% of the post-fee distributable pot, not 80% of gross sales. With 800.00 USDC gross, that distinction produces 608.00 USDC for the winner, not 640.00 USDC.

FIGURE 14 **Pixel Passport #42 across all branches**



Illustrative names and assets; every amount is computed from compiled constants in six-decimal raw units.

All names and assets are fictional. Every monetary amount is produced by the build from compiled constants and six-decimal integer arithmetic.

PART VII

Failed-Draw Refunds

The final reviewed implementation uses transferable bearer tickets that burn for refunds. It does not snapshot owners or credit refunds in a separate batch.

PART VII

Failed-Draw Refunds

35. Why Refund Redemption Is Bounded

A raffle may sell many tickets. Processing every ticket in one transaction would create a loop whose gas cost grows with total sales. At some size, the transaction could become too expensive or impossible to include.

The reviewed contract solves this by letting each actual owner submit 1 through 100 ticket IDs per call. Each call performs bounded ownership checks, burns, liability reduction, and one exact USDC transfer.

This is not permissionless refund crediting. An unrelated keeper cannot pre-credit a refund to a frozen owner. The owner of every listed ticket must be the caller.

36. Who Owns the Refund?

The refund belongs to the current owner of an unburned refundable ticket. Ownership is evaluated when `redeemRefundTickets` executes.

If Maya buys ticket 12, transfers it to Leo before `Refunding`, and Leo transfers it to Noor after `Refunding`, Noor may burn it for 10.00 USDC. Maya and Leo have no stored refund claim for that ticket. The bearer right moved with the unburned NFT.

This rule is simple but operationally important. Do not transfer a refundable ticket unless you intend to transfer its unconsumed refund right.

37. Redeeming Refunds

FIGURE 15 **Refund redemption**

The actual owner supplies a bounded list, each ticket burns, the aggregate liability falls, and exact USDC moves to the chosen destination in one reverting transaction.

The caller supplies a list of ticket IDs and a nonzero destination. The function requires `Refunding`, a list length from 1 through 100, and actual caller ownership of every ID.

For each ticket, the raffle reads `ownerOf`, rejects a foreign owner, and burns the token. It then calculates:

```
refundAmount = ticketPrice x ticketIds.length
```

The function subtracts that amount from `remainingRefundLiability` and performs one exact outgoing USDC transfer.

Duplicate IDs, already burned IDs, nonexistent IDs, foreign tickets, a zero destination, an oversized list, or an unsupported USDC transfer revert the complete transaction. Earlier tentative burns and liability changes in that call are restored.

38. Receiving the Refund

The ticket owner may choose a different nonzero USDC destination. This is useful if the ownership wallet is blacklisted, a contract wallet needs funds elsewhere, or an operator separates signing and custody.

Choosing another destination does not let someone else redeem the ticket. The caller must still be the actual owner. The outgoing token must debit the raffle and credit the destination by exactly the expected amount.

No separate pull claim remains after a successful refund transaction. The ticket burn and USDC transfer succeed together.

39. Ticket Transfers During Refunds

Tickets remain ordinary ERC-721 bearer tokens in `Refunding`. A transfer before burn moves the refund right. A transfer and a competing refund transaction may race; the first valid transaction included changes or consumes ownership, and the other must re-evaluate or revert.

After redemption, the ticket is burned. It cannot transfer as a souvenir and its metadata URI is no longer returned by `tokenURI` because the token no longer exists.

Known protocol destinations remain rejected. Arbitrary user-selected incapable contracts remain a risk. If an unsafe transfer locks a refundable ticket in such a contract, `remainingRefundLiability` may remain forever because there is no generic administrator rescue.

40. Refund Conservation

Immediately after refund finalization:

`remainingRefundLiability = grossSales`

After any successful refund redemption:

`USDC already refunded + remainingRefundLiability = grossSales`

Equivalently, for `n` burned refund tickets:

`remainingRefundLiability = grossSales − (n x ticketPrice)`

The broader contract identity is:

`accountedQuoteBalance = unsettledPot + remainingRefundLiability + winnerCashLiability + totalClaimableQuote`

Only one branch should carry each unit. Direct USDC donations can make the contract's actual balance greater than this accounting identity, but cannot change the identity or create a claim.

41. Why Failed Draws Charge No Fee

The protocol fee is created only inside a valid Entropy callback that reaches `NftWon` or `CashWon`. `enableRefunds` does not calculate or credit a fee.

The design treats missing request and missing callback as failure to complete the paid random outcome. The gross ticket pot remains dedicated to exact ticket refunds. The sponsor receives no quote proceeds and the treasury receives no fee, while the fixed recovery recipient may recover the NFT.

A LIABILITY IS NOT A FORCE FIELD

Exact accounting preserves what the contract owes. It cannot make a paused or blacklisted USDC contract transfer, make a trapped bearer call redemption, or guarantee Base inclusion.

PART VIII

Claims and Custody

Sponsor and treasury proceeds are pull claims. Winning and refund rights are bearer redemptions. Prize recovery is a fixed-recipient claim.

PART VIII

Claims and Custody

42. Why Some Payments Are Pulled

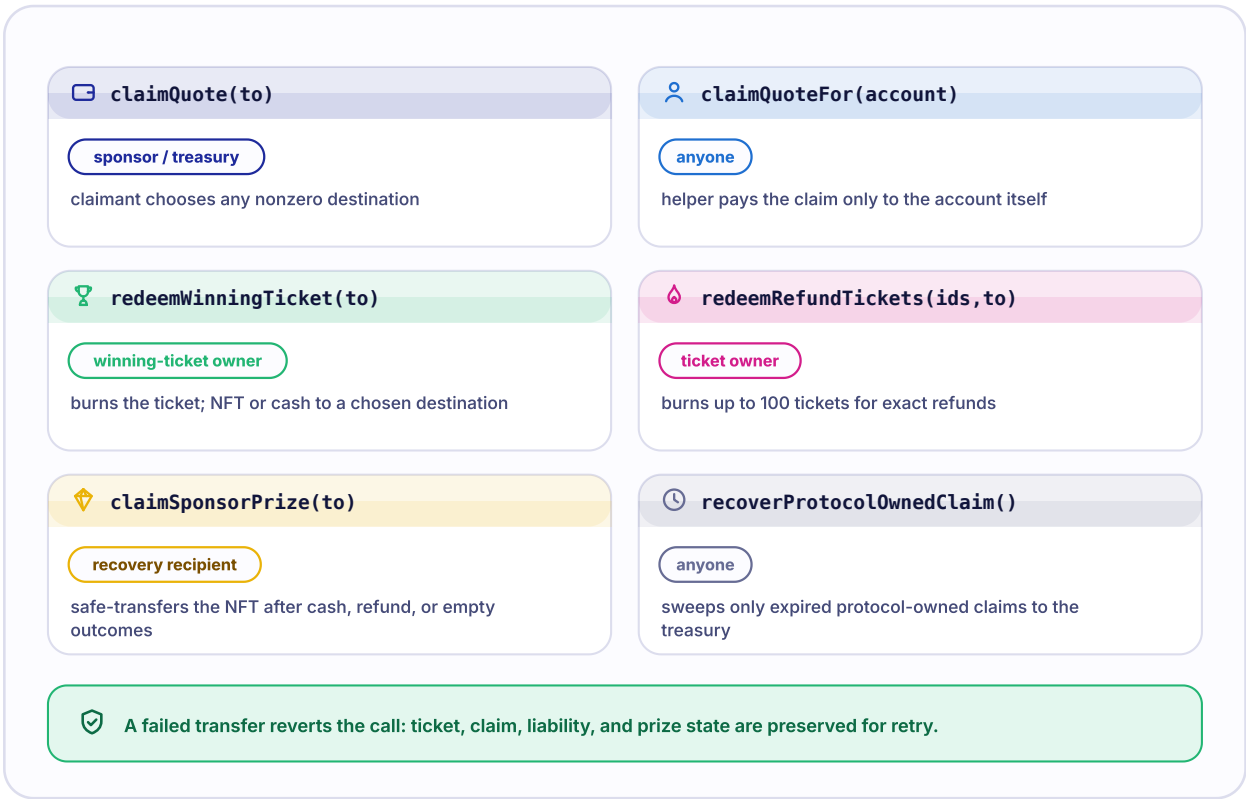
Pushing every asset during the randomness callback would couple settlement to external token contracts, prize receivers, sponsor wallets, and treasury wallets. One failure could block the callback or consume unpredictable gas.

raffle.fun makes the callback storage-only. It records the selected ticket and quote liabilities. Recipients act later and independently.

The exact model is mixed:

- sponsor and treasury proceeds are pull claims;
- NFT-winning and cash-winning rights are consumed by burning the selected ticket;
- refunds are consumed by burning bounded batches of tickets;
- sponsor-side prize recovery is initiated by the fixed recipient;
- native Entropy overpayment is returned immediately, not pulled later.

FIGURE 16 **Claims and bearer redemptions**



Each asset path has a specific claimant and destination rule. A failed transfer reverts without consuming the right.

43. Quote Claims

`claimQuote(to)` pays the caller's complete `claimableQuote[msg.sender]` balance to a chosen nonzero destination. Only sponsor and treasury amounts are normally credited to this mapping.

Before transfer, the contract clears the individual and aggregate liability. It then requires exact raffle debit and exact recipient credit. A revert restores the cleared claim because the whole transaction is atomic.

The function rejects payment to the raffle itself. It also rejects a zero destination and an account with no claim.

44. Claim-for Quote Payments

`claimQuoteFor(account)` may be called by anyone, but it hard-codes the destination to `account`. A keeper can help a sponsor or treasury submit a claim without gaining the ability to redirect funds.

This helper is useful for ordinary externally owned accounts and callable contract wallets. A malicious, blacklisted, paused, or self-referential destination may still fail. Protocol creation checks and the selective protocol-owned recovery helper cover known same-factory cases, not every arbitrary address.

45. Winning Prize Redemption

In `NftWon`, the current owner calls `redeemWinningTicket(to)`. The contract confirms actual ownership, burns the ticket, marks the prize claimed, and safely transfers the NFT to the chosen nonzero destination.

The caller may not be only an approved operator. It must equal `ownerOf`. A rejecting destination reverts the entire operation and restores the ticket and prize claim.

In `CashWon`, the same function burns the selected ticket and exact-transfers `winnerCashLiability` to the chosen destination. It does not transfer the NFT.

There is no general `claimPrizeFor` function. No third party can force a winning bearer's prize to the bearer address. The bearer must initiate redemption or transfer the ticket to a capable owner.

46. Sponsor-Side Prize Recovery

`claimSponsorPrize(to)` is available only to the immutable `sponsorPrizeRecoveryRecipient` in `CashWon`, `Refunding`, or `Closed`. It marks `prizeClaimed` and safely transfers the configured NFT to the chosen nonzero destination.

The sponsor cannot call unless it is also the fixed recovery recipient. The factory owner, treasury, frontend, and refund finalizer cannot call on the recipient's behalf.

A same-factory raffle that owns a prize right through a predicted future address has a narrow permissionless recovery path. Any caller can invoke the holding raffle's `recoverProtocolOwnedClaim`, but it can target only a registered raffle, select only one of four fixed claim kinds, and route recovered assets only to the holder raffle's immutable recovery recipient.

47. Native-Fee Overpayment

The draw requester pays Base-native currency to `requestDraw`. The raffle forwards exactly Pyth's quoted fee and immediately returns excess native currency to the requester.

There is no `claimNative`, native liability mapping, or later overpayment withdrawal. If the requester rejects the immediate return, the whole request reverts.

Direct native transfers to the raffle revert. Native value can still be forced by EVM mechanisms outside an ordinary call. Forced value is unaccounted surplus and creates no claim.

48. Direct Donations and Surplus

Direct USDC donations and forced native value:

- do not increase tickets or gross sales;
- do not change the selected winner;
- do not change fee, sponsor, winner, or refund arithmetic;
- do not create a claim for the donor;
- appear as value above protocol accounting;
- cannot be swept by the factory owner.

The no-rescue choice reduces administrator seizure power but can leave surplus or unrelated assets permanently inaccessible.

49. Claims That Can Still Fail

A valid onchain right can still be hard or impossible to realize:

- USDC may pause, blacklist, upgrade, or report dishonest balances;
- a prize contract may reject safe transfer, burn the token, reenter, or change code;
- a chosen contract destination may reject ERC-721 receipt;
- a bearer ticket may be unsafe-transferred to an incapable arbitrary contract;
- the claimant may lose keys or use a broken wallet;
- the Base sequencer or network may delay or censor transactions;
- gas costs may exceed a user's available native currency.

Atomic reverts preserve contract state when an external interaction fails. They do not make the external dependency available.

PART IX

Contract Architecture

The protocol has a future-policy factory, independent constructor-deployed raffles, and a read-only registry-authenticated Lens, plus offchain access layers.

PART IX

Contract Architecture

50. RaffleFactory

`RaffleFactory` is a non-upgradeable `Ownable2Step` registry and constructor deployer. Its constructor fixes:

- one quote-token contract;
- one Pyth Entropy v2 contract;
- one nonzero callback gas limit;
- an initial protocol treasury;
- an initial owner.

The factory stores a mutable creation pause and treasury for future raffles, a raffle counter, ID-to-address and address-to-ID mappings, and the canonical `isRaffle` registry.

`createRaffle` validates terms, deploys, registers, emits, deposits, and verifies in one nonreentrant transaction. The factory does not custody ticket proceeds or prizes after creation.

51. Raffle Deployments

Each `Raffle` is an ordinary `CREATE` deployment with constructor-fixed configuration. It is not an EIP-1167 clone, a proxy, or upgradeable. It inherits OpenZeppelin's ordinary `ERC721`, not `ERC721Upgradeable`.

Every raffle independently stores its status, ticket count, gross sales, four quote- accounting components, Entropy sequence and request time, winning ticket, prize claim flag, metadata URI, and sponsor/treasury quote claims.

The raffle has no owner, administrator, pause, implementation pointer, generic call, or asset sweep. Factory ownership does not propagate into it.

52. RaffleLens

`RaffleLens` is read-only. Its immutable factory identifies canonical raffles. Before forwarding any read, the Lens checks `factory.isRaffle(raffle)`.

The Lens combines lifecycle, assets, timestamps, liabilities, winning ownership, dynamic Entropy fee availability, and account-specific action flags. A temporary Entropy fee-read failure is reported as unavailable rather than hiding all other state.

`getRaffleStates` accepts at most 64 addresses. The Lens owns no assets and changes no state. A Lens result is a convenience snapshot at one block, not a transaction guarantee.

53. Pyth Entropy Dependency

Each raffle permanently stores the factory's Entropy address and callback gas limit. The same gas limit is used for `getFeeV2` and `requestV2`. A changed Pyth fee is read at request time.

Deployment validation must verify the official chain-specific Entropy address and measure the callback gas limit against exact deployment bytecode. Fork tests are evidence about pinned historical blocks, not proof that a future address or provider policy remains correct.

54. SDK

The TypeScript SDK synchronizes ABIs from Hardhat artifacts. Its write helpers simulate contract calls before sending them through a wallet. It covers creation, purchases, draw request, refunds, empty close, winner redemption, sponsor prize recovery, quote claims, and protocol-owned recovery.

The SDK validates refund lists for nonempty, positive, unique, bounded ticket IDs before simulation. SDK checks improve usability but cannot override chain state or make a failing token transfer succeed.

55. Subgraph

The subgraph listens to factory creation and raffle events. It reconstructs sponsors, recovery recipients, tickets, transfers, burns, request data, status, fees, quote claims, remaining refund liability, and prize claims.

An indexer can lag, omit a block, use the wrong deployment, or process a reorganization. The subgraph is for discovery and history. It is never transaction or settlement authority.

56. Frontend

The web application discovers raffles through indexed data, then reads authoritative state and simulates writes. Missing or wrong-chain deployment configuration disables writes. The interface presents creation, purchase, draw, refund, winner, recovery, and quote-claim actions.

The checked-in sandbox is a preview model, not a deployment and not proof of contract behavior. A website can also be compromised. Users should verify the chain, contract address, function, value, and destination in the wallet.

57. Onchain Versus Indexed State

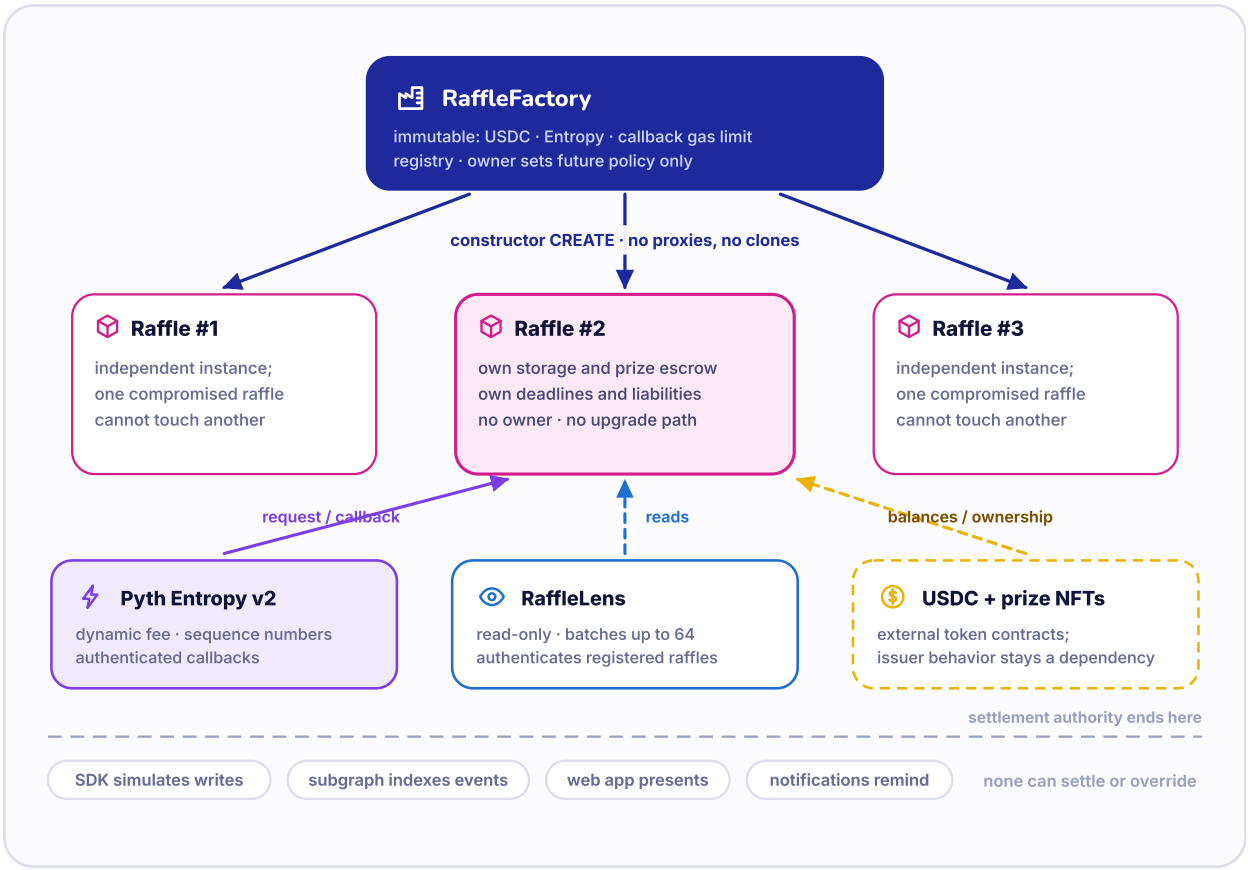
When data disagrees, use this priority:

1. production Solidity and current onchain state;
2. executable contract tests and generated ABI behavior;
3. a factory-authenticated Lens read;
4. SDK simulations and direct RPC reads;
5. subgraph entities;
6. frontend labels and cached views;
7. prose documents.

The lower layers remain useful, but only the EVM state transition determines whether a transaction succeeds.

58. Architecture Diagram

FIGURE 17 **Contract architecture**



The factory deploys independent raffles. The Lens and offchain layers read state but do not settle assets. Pyth, USDC, and prize contracts remain external dependencies.

PART X

Administration and Immutability

Factory administration affects only future creation. Existing raffles have no owner, upgrade, rescue, pause, or settlement override.

PART X

Administration and Immutability

59. What the Factory Owner Can Do

The factory owner has three categories of power:

1. call `setCreationPaused` to pause or resume new creation;
2. call `setProtocolTreasury` to change the treasury captured by raffles created later;
3. begin, accept, or renounce OpenZeppelin ownership according to `Ownable2Step` and inherited `Ownable` rules.

Treasury updates reject zero, the factory, the quote token, Entropy, and already registered raffles. A future code-less raffle address can create a narrower issue; the selective protocol-owned recovery helper exists for that same-factory case.

The checked-in Ignition module deploys the factory with the deployer as initial owner, deploys the Lens, and begins transfer to a configured final owner. No verified deployment proves that a final multisig has accepted ownership.

60. What the Factory Owner Cannot Do

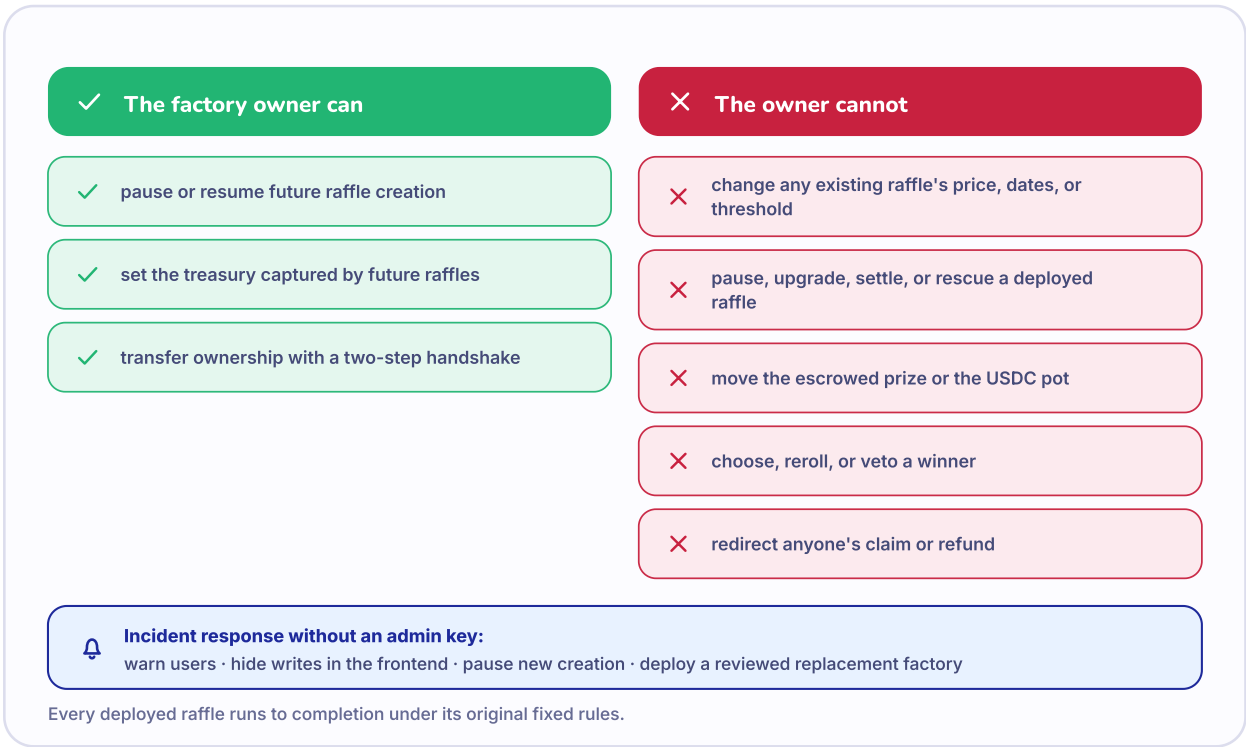
The factory owner cannot change an existing raffle's:

- ticket price, minimum threshold, sale dates, or metadata;
- sponsor, recovery recipient, treasury, quote token, prize, Entropy, or callback gas;
- ticket ownership, winning ticket, status, fee, payout, refund, or claim;
- prize custody or quote balance.

The owner also cannot:

- pause an existing sale or settlement;
- request replacement randomness or a second draw;
- select a winner;
- enable an early refund after sales;
- redirect an existing claim;
- seize or rescue a prize;
- sweep direct donations or unrelated NFTs;
- upgrade an existing raffle.

FIGURE 18 **Factory owner can and cannot**



Administration changes future creation policy. It does not create an emergency control plane for deployed raffles.

61. Why Existing Raffles Cannot Be Patched

An independently deployed raffle has no implementation pointer. Its runtime bytecode and constructor immutables do not consult a mutable factory implementation. This prevents an owner from changing the rules after users enter.

The cost is irreversible code risk. If a deployed raffle contains a defect, maintainers cannot patch, pause, migrate, or rescue it through an administrator. Users must rely on the existing functions, supported asset behavior, and Base inclusion.

62. Operational Incident Response

Maintainers can pause new creation, disable first-party writes, warn users, investigate, publish contract-level instructions, monitor deadlines, and deploy a separately reviewed new factory version. They cannot rewrite existing storage or promises.

An incident plan therefore needs:

- clear deployment records and network labels;
- monitoring for request, callback, timeout, liabilities, and failed claims;
- user communication outside the frontend;
- a process for independent disclosure and external review;
- a new-factory migration plan that never implies old raffles were upgraded;
- legal and sanctions escalation paths.

No production monitoring, final Safe policy, or deployment runbook is verified as complete at the reviewed commit.

PART XI

Security and Trust

The internal campaign exercises custody, accounting, randomness, recovery, and integration properties. It is not an independent external audit or formal proof.

PART XI

Security and Trust

63. Security Goals

The principal internal security objective is conditional:

For an honest standards-compliant ERC-721 prize whose ownership and safe transfers remain available, and an exact-transfer non-rebasing USDC whose balance reporting and transfers remain available, every protocol-controlled lifecycle should expose a bounded path for the authorized bearer or fixed recipient to claim the configured prize and every accounted quote liability.

Additional goals include one accepted randomness request, one terminal economic branch, exact quote conservation, no administrator settlement override, atomic creation, callback-only storage work, and bounded loops.

64. Prize-Custody Protections

The receiver authenticates the exact prize, sponsor, factory operator, and construction status. The factory verifies post-transfer ownership and activation. Winner redemption burns the credential before safe transfer, and sponsor-side recovery marks the prize before transfer. A revert restores all effects.

The prize should leave escrow at most once through one of two authorities: the winning bearer in `NftWon`, or the fixed recovery recipient in `CashWon`, `Refunding`, or `Closed`.

These controls cannot make malicious prize code honest or recover unrelated NFTs forced into the raffle.

65. Quote-Solvency Protections

Inbound payment must credit the exact gross amount. Outbound payment must create the exact raffle debit and recipient credit. Every successful branch allocates all raw units. Every failure branch moves the gross pot into exact refund liability.

The contract exposes:

```
accountedQuoteBalance = unsettledPot + remainingRefundLiability + winnerCashLiability + totalClaimableQuote
```

Tests assert the actual supported-token balance never falls below this value. A direct donation can create surplus, but cannot dilute or enlarge liabilities.

66. Randomness Protections

The configured Entropy wrapper authenticates callback entry. The raffle checks the stored sequence, `Drawing` status, and in-flight guard. One request is accepted. Wrong, duplicate, early, stale, and late callbacks do not settle.

The winner formula includes the complete sold range. Request and callback failures retain deterministic routes to `Refunding`. The callback does no token or user calls.

These controls do not independently verify the oracle's random value, prevent provider and validator collusion, eliminate modulo bias, or defeat Base ordering.

67. Refund Protections

One `enableRefunds` function covers both missing-request and callback-timeout failure. The transition is permissionless only after the applicable exact deadline. It charges no fee and moves the complete unsettled pot into one refund liability.

Each refund call accepts at most 100 actual owner-held ticket IDs. Each valid ID burns once for one ticket price. Duplicate, foreign, invalid, and already burned IDs revert. A failing USDC transfer restores the burns and liability.

68. Reentrancy Protections

The factory and every asset-moving raffle action use `ReentrancyGuard`. The contract also follows checks, effects, and interactions: state and liabilities are updated before external transfers, and a revert restores them.

The internal review includes reentrant quote tokens, prize contracts, ticket receivers, and factory reentry during prize deposit. Passing those tests does not enumerate every external-contract behavior.

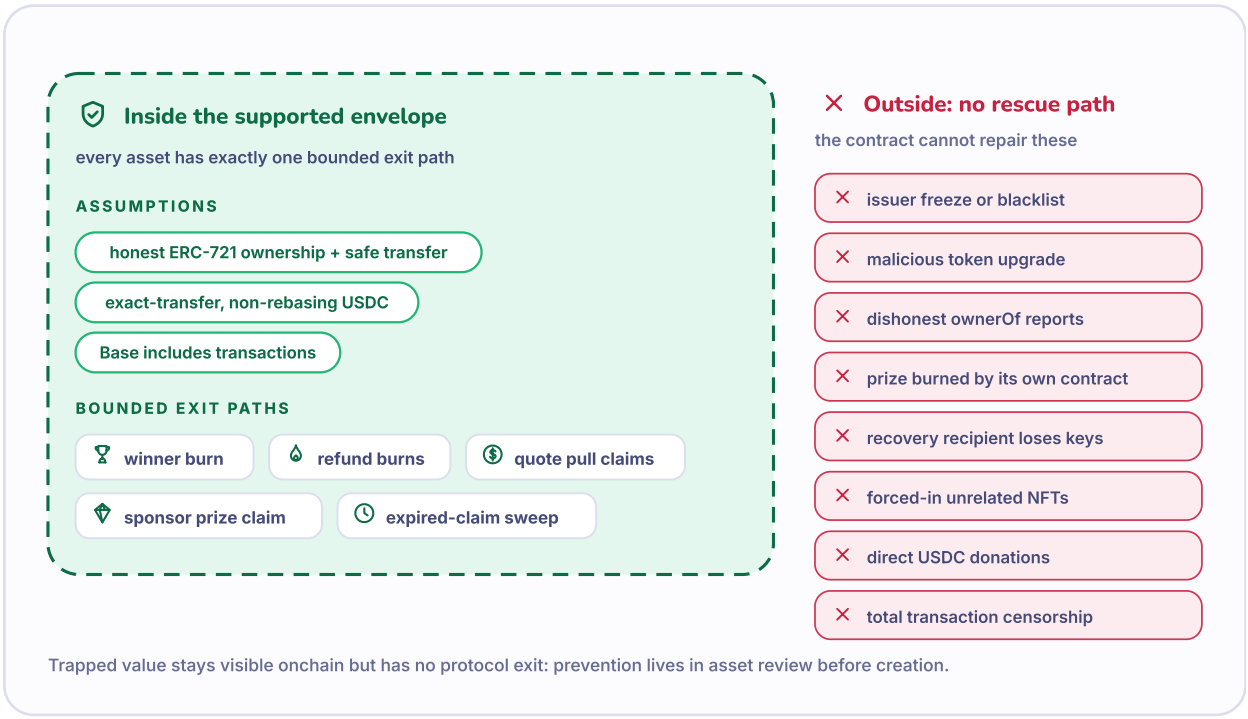
69. Bounded-Execution Protections

One purchase mints at most 100 tickets. One refund redemption burns at most 100 tickets. The Lens reads at most 64 raffles. The callback performs constant-sized storage work.

The internal gas ledger measured 95,078 gas for callback storage work against the 300,000 test configuration, 601,290 gas for the worst observed 100-refund branch, and 2,803,881 gas for a 100-ticket contract-receiver purchase. These are measured results, not guarantees for a future chain environment.

70. Supported Recovery Envelope

FIGURE 19 **Supported recovery envelope**



The internal recovery property covers protocol paths under honest asset and chain assumptions. External freezes, malicious code, lost keys, and universal censorship remain outside it.

Inside the supported envelope:

- the prize reports ownership honestly and safe transfers remain available;
- USDC is exact-transfer, non-rebasing, and available;
- Base continues executing and including transactions;
- authorized accounts can submit calls or transfer bearer rights;
- the configured Pyth address and callback authentication are correct.

Outside the envelope:

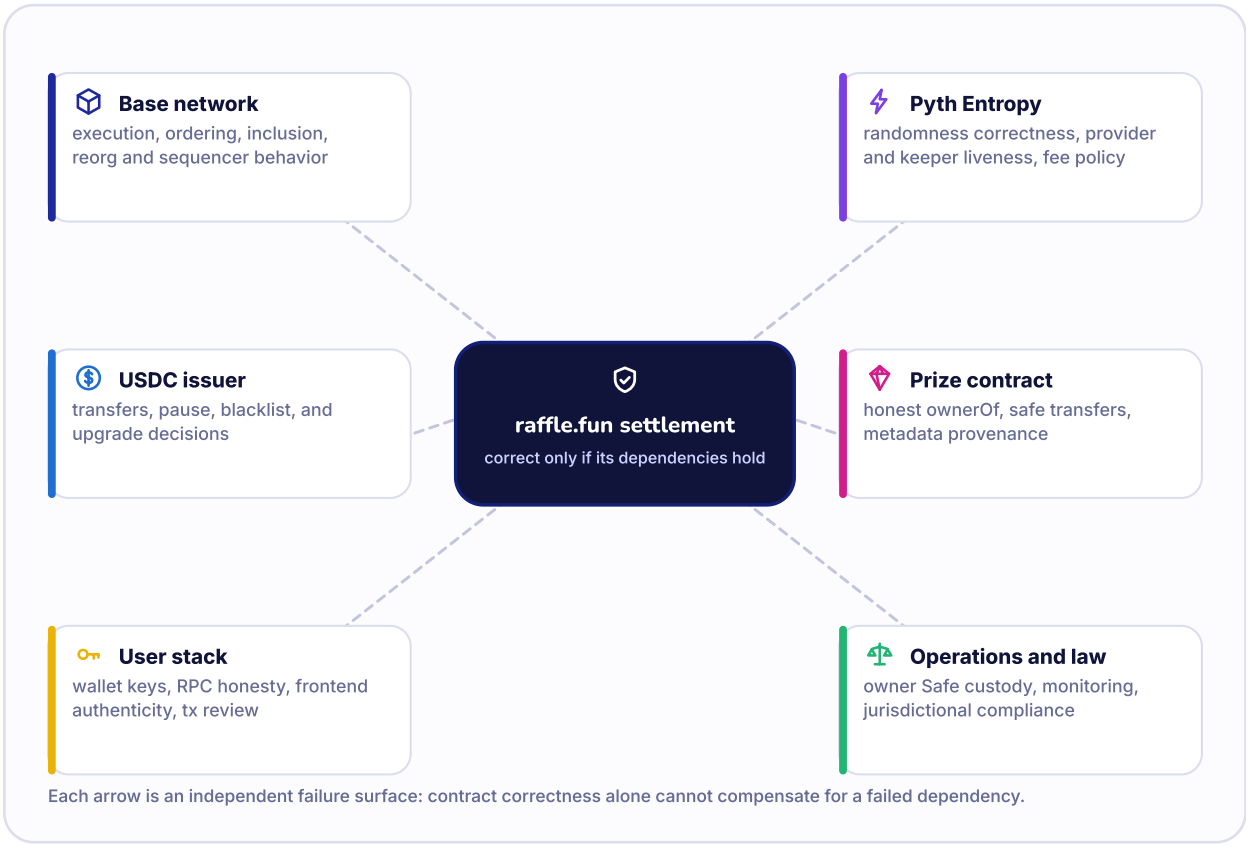
- issuer freeze, blacklist, pause, upgrade, or dishonest balance reporting;
- malicious or upgraded NFT behavior, burned prize, or false `ownerOf`;
- arbitrary incapable recipient contracts and lost keys;
- unrelated NFTs or unaccounted donations;

- chain halt, universal censorship, or severe reorganization;
- legal prohibition or operational misuse.

The no-rescue design is part of the trust model, not proof every asset is recoverable.

71. Threat and Dependency Map

FIGURE 20 **Threat and dependency map**



Onchain logic is only one layer. Base, Pyth, asset issuers, wallets, RPCs, metadata, operations, and law each create independent failure modes.

DEPENDENCY	WHAT RAFFLE.FUN RELIES ON	RESIDUAL RISK
Base	EVM execution, timestamps, transaction ordering and inclusion	sequencer delay or censorship, reorg, halt, fee volatility
Solidity compiler	correct 0.8.36 compilation for pinned settings	compiler defect or mismatched build
OpenZeppelin 5.6.1	ERC-721, safe token helpers, ownership, reentrancy guard, math	dependency defect or version drift
Pyth Entropy v2	fee quote, request, authenticated callback, random correctness	provider, keeper, address, fee, collusion, availability
USDC issuer	exact balances and continuing transfers	pause, blacklist, upgrade, policy change
Prize contract	truthful ERC-165 and ownership, available safe transfer	malicious code, burn, reentry, metadata change
Wallet and RPC	correct signing, chain, address, and transaction broadcast	compromise, phishing, stale or censored access
Frontend and subgraph	usable display and discovery	stale, wrong-chain, compromised, non-authoritative data
Factory-owner account	disciplined future-only policy	creation disruption or unsafe future treasury selection
Metadata host	descriptive ticket and prize content	disappearance, replacement, false description

72. Internal Testing and Review

The internal review identity was baseline commit `a2120f5e163dc3641d9864773febbfedca047edb` plus patch fingerprint `55589ce1fce0bd5579e60df747796575df9b0d96`. The final reviewed implementation was later committed as `f165e4c1d8f5d093fe0a36094f79a29857c26286`. This paper reviews that committed result.

CATEGORY	TOOL AND VERSION	COMPLETED EVIDENCE	RESULT
Default Solidity suite	Forge 1.2.3-stable; solc 0.8.36	70 passed, 0 failed, 1 RPC-gated fork suite skipped	pass
Critical fuzz	Foundry	6 properties x 100,000 plus 6 × 10,000 second seed	pass
Differential model	Foundry	10,000 sequences, 1-32 actions, terminal drain	pass
Stateful invariants	Foundry	1,000 × 256 multi-actor; 1,000 × 256 strict; 5,000 x 512 release strict	pass
Strict handler calls	Foundry	10,240,000 release calls, zero reverts	pass
Independent fuzzers	Echidna 2.3.3; Medusa 1.5.1	200,421 and 113,261 transactions/calls	pass
Mutation	Gambit 1.0.6	52 compiling mutants, 52 killed, 100% sample score	pass
Focused symbolic	Halmos 0.3.3; Z3 4.12.6	5 focused properties	pass
Whole-graph symbolic	SMTChecker	unsupported dependency features; no embedded whole-graph assertions	inconclusive
Mythril	0.24.8 candidate	unavailable for installed Python 3.12 isolation	not executed
Static analysis	Slither 0.11.6; Aderyn 0.6.8; Semgrep 1.170.0; Solhint 6.2.3	Slither zero final findings; other patterns manually classified	pass with manual classification
Production coverage	Foundry	100% lines and functions; 98.84% branches; anchor warning documented	pass
Gas	Foundry snapshot and targeted tests	callback, 100-ticket, 100-refund, claims, factory, and Lens measured	pass
Fork checks	Foundry against pinned Base blocks	Base 49,752,968 and Base Sepolia 45,263,498; 2 tests	pass
Hardhat	Hardhat 3.11.1	5 deployment and lifecycle tests	pass
SDK	Vitest	10 tests and generated ABI sync	pass
Subgraph	Matchstick	4 mapping tests, codegen and build	pass
Web	ESLint, TypeScript, Vitest, Next	lint, typecheck, 14 tests, production build	pass

The fork suite used real historical USDC and Pyth deployments but did not impersonate Pyth to prove a real production callback. No public or testnet transaction was broadcast.

73. Remaining Findings

The internal campaign reproduced four High-severity protocol-self custody failures, one Medium-severity fee-destination failure, and two Low release-process failures. All were remediated and preserved as regressions. No unresolved Critical, High, or Medium supported-asset finding remained in the internally reviewed final source.

One Informational risk is accepted: an owner may unsafe-transfer a bearer ticket to an arbitrary unrelated contract that cannot initiate redemption. Known protocol destinations are rejected, and future same-factory raffle destinations have a selective recovery helper, but arbitrary bytecode capability cannot be solved without removing transferability or adding generic rescue power.

The factory runtime measured 24,311 bytes, only 265 bytes below the EIP-170 limit. Any production Solidity change requires a fresh size and security campaign.

74. What Testing Does Not Prove

Passing tests does not prove absence of unknown bugs. Fuzzing samples inputs. Stateful invariants explore configured sequences. Mutation measures one generated mutant sample. Coverage shows execution, not correctness. Static tools have false positives and false negatives. Focused symbolic checks prove only their modeled paths.

No independent external audit has been completed. No monitored Base Sepolia lifecycle campaign has been completed. Whole-protocol formal verification has not been completed. Legal and operational review have not been completed.

75. What the Protocol Cannot Guarantee

raffle.fun cannot guarantee:

- legal availability or compliance for a sponsor, participant, or jurisdiction;
- value, authenticity, metadata, intellectual-property rights, or liquidity of an NFT;
- USDC transfer availability or continued value;
- perfect, trust-free, or bias-free randomness;
- Base uptime, ordering neutrality, finality timing, or universal transaction inclusion;
- safety of wallet keys, RPCs, browsers, frontends, or user destinations;
- recovery from malicious external assets, lost keys, or unrelated forced transfers;
- absence of unknown smart-contract, compiler, dependency, or integration defects.

Security review status: Internal adversarial review only. Independent external audit: Not completed.

PART XII

User Guides

These guides describe contract actions, not a currently authorized deployment. Verify network, addresses, legal status, token behavior, and independent review first.

PART XII

User Guides

76. Sponsor Walkthrough

1. **Do not use an unverified deployment.** Deployment status in this paper is Not deployed and not authorized for user funds.
2. **Obtain jurisdiction-specific advice.** The protocol does not determine whether the proposed chance-based promotion is legal.
3. **Review the prize.** Confirm ERC-721 address, token ID, ownership, approval, transfer behavior, upgrade controls, metadata, and rights.
4. **Review factory dependencies.** Confirm Base chain ID, canonical factory, factory- wide USDC, Pyth Entropy address, callback gas, treasury, owner, and Lens.
5. **Choose terms.** Set gross price, nonzero threshold, inclusive start, exclusive end, and bounded metadata.
6. **Choose recovery.** Verify the fixed sponsor-side prize-recovery recipient. Zero defaults to the sponsor.
7. **Approve the factory for the exact NFT.** Avoid broad approvals when a narrower approval is practical.
8. **Simulate and create.** Confirm the transaction deploys, registers, deposits, and verifies atomically.
9. **Verify the result.** Read the `RaffleCreated` event, registry ID, raffle immutables, `Active` status, and prize `ownerOf`.
10. **Monitor the lifecycle.** Watch sale end, request grace, request sequence, callback deadline, status, and quote claims.
11. **Claim the applicable assets.** In successful outcomes, use `claimQuote`. In cash, refund, or empty outcomes, have the fixed recovery recipient call `claimSponsorPrize`.

Once any ticket has sold, the sponsor cannot cancel, change terms, pause the raffle, or take the prize back by discretion.

77. Ticket-Buyer Walkthrough

1. Verify the Base network and contract address independently of a link or social post.
2. Confirm the raffle is factory-registered and read current state from chain.
3. Verify the prize contract and token ID, but remember custody does not prove value or metadata authenticity.
4. Read ticket price, threshold, start, end, request deadline, recovery recipient, and quote token.
5. Understand the branches: threshold missed after a valid callback means one cash winner, not general refunds.
6. Approve only enough USDC for the intended gross purchase when practical.
7. Confirm quantity and recipient. The payer need not become the ticket owner.
8. Submit `buyTickets` and verify the receipt and minted ticket IDs.
9. Track current ownership. Transfer moves the winning or refund right until burn.

10. Keep enough native currency for later gas. A refund right does not pay gas.

78. Draw-Requester Walkthrough

Alex or any account may request after `endTime` and before the grace deadline if at least one ticket exists.

1. Read `status`, `totalTickets`, `endTime`, `requestGraceDeadline`, and current time.
2. Call `getEntropyFee` close to submission because the fee can change.
3. Use an account that can accept an immediate native overpayment return, or send the exact fee.
4. Simulate `requestDraw` with the intended native value.
5. Submit and verify `DrawRequested`, the stored sequence, `Drawing`, request time, and callback deadline.
6. Do not expect reimbursement or an economic reward from the ticket pot.

Only one request may succeed. A failed attempt before the deadline does not consume the opportunity if the complete transaction reverts.

79. Refund-Finalizer Walkthrough

There is no separate permissionless refund-credit executor in the final implementation. There are two roles:

- **failure finalizer:** anyone may call `enableRefunds` after the correct deadline;
- **refund redeemer:** the actual ticket owner burns owned tickets for payment.

To finalize failure:

1. read status and the applicable deadline;
2. confirm tickets exist;
3. at or after the deadline, simulate `enableRefunds`;
4. submit and verify `RefundsEnabled`, `Refunding`, zero `unsettledPot`, and the full `remainingRefundLiability`;
5. publicize direct contract instructions so owners can redeem.

The finalizer cannot choose owners, receive a fee, or redirect refunds.

80. Winner Walkthrough

1. Verify `NftWon` or `CashWon`, `winningTicketId`, and that your address is the actual current `ownerOf`.
2. Treat transfer approval as insufficient; the redemption caller must be the owner.
3. Choose a destination capable of receiving the asset. For NFT outcome, a contract destination must accept safe ERC-721 transfers. For cash, USDC must be available to that destination.
4. Simulate `redeemWinningTicket(to)`.
5. Submit and verify the ticket burn and asset movement.

If the transfer fails, the transaction should revert and preserve the unburned ticket and liability. Diagnose the destination or external token before retrying.

81. Refund-Recipient Walkthrough

1. Verify `Refunding` and list only ticket IDs you currently own.
2. Split large holdings into batches of at most 100.
3. Remove duplicates and already burned IDs.
4. Choose a nonzero USDC destination.
5. Simulate `redeemRefundTickets(ids, to)`.
6. Submit and verify one burn per ID, exact payment, and reduced remaining liability.

Transferring an unburned refundable ticket transfers its refund right. After burn, the ticket no longer exists.

82. Safety Checklists

Sponsor checklist

- [ ] Jurisdiction-specific gambling, lottery, sweepstakes, contest, consumer, sanctions, tax, licensing, advertising, and age rules reviewed.
- [ ] Official Base chain and deployment record independently verified.
- [ ] Independent external audit status checked for the exact source and locks.
- [ ] Prize address, token ID, owner, metadata, upgrade controls, and transfer behavior reviewed.
- [ ] Factory-wide USDC and issuer risks reviewed.
- [ ] Price, threshold, start, end, and metadata confirmed.
- [ ] Fixed recovery recipient confirmed and able to call.
- [ ] Factory approval limited and transaction simulated.
- [ ] Atomic escrow and `Active` state verified after creation.
- [ ] Monitoring and claimant gas plans prepared.

Buyer checklist

- [ ] Official network, factory registration, and raffle address verified.
- [ ] Prize and quote-token contracts independently checked.
- [ ] Gross price, quantity, recipient, allowance, and wallet transaction reviewed.
- [ ] Threshold understood as branch selector, not sales cap.
- [ ] Cash fallback understood as one winner with no general refunds.
- [ ] Request and callback deadlines understood.
- [ ] Ticket transfer understood to move all unconsumed bearer rights.
- [ ] External token, oracle, chain, wallet, and legal risks accepted.
- [ ] Enough native currency retained for gas.
- [ ] No more value committed than the user can afford to lose.

PART XIII

Frequently Asked Questions

Short answers to the most important custody, ownership, randomness, refund, claim, administration, audit, deployment, and legal questions.

FAQ Custody, tickets, outcomes, claims, and risk

PART XIII

Frequently Asked Questions

1. Who holds the NFT during the raffle?

The independently deployed raffle contract owns the configured prize after atomic creation succeeds and until a supported claim transfers it out.

2. Can the sponsor take the NFT back after a ticket sells?

No discretionary path exists. After a sale, the raffle must follow successful randomness or a deadline-based refund branch. The fixed recovery recipient gets the NFT only in cash, refund, or empty outcomes.

3. Can the sponsor cancel before sales?

The sponsor may call `closeEmptyRaffle` before `endTime` only while zero tickets have ever sold. There is no general cancellation status or function.

4. Can the factory owner choose the winner?

No. The winner is calculated from the authenticated Pyth value and sold-ticket count.

5. Can the factory owner change an existing raffle?

No available owner function changes its price, threshold, dates, assets, parties, status, winner, liabilities, or claims.

6. Is the minimum-ticket threshold a cap?

No. It selects NFT versus cash after a successful callback. Sales continue above it until the exclusive end.

7. What happens exactly at the threshold?

Equality meets the threshold and produces `NftWon` after a valid callback.

8. Can tickets transfer?

Yes. Tickets remain transferable in every status until burned, subject to rejection of known protocol destinations.

9. Who wins after a ticket transfer?

The actual current owner of the selected unburned ticket may redeem. Ownership can change even after selection.

10. When do ticket transfers freeze?

They do not freeze in the final reviewed implementation.

11. What happens if nobody buys a ticket?

The sponsor may close early, or anyone may close at or after sale end. The fixed recovery recipient then claims the NFT.

12. What happens if randomness is never requested?

At `endTime + 3 days`, anyone may enable refunds. There is no winner or fee.

13. What happens if the callback never arrives?

At `drawRequestedAt + 2 days`, anyone may enable refunds if a valid callback has not already settled.

14. Can a callback still win at the timeout boundary?

Yes, before timeout finalization changes status. Callback and timeout may race; the first valid included transaction determines the terminal outcome.

15. Are failed-draw refunds automatic?

No. Anyone first enables `Refunding`, then each actual ticket owner burns tickets in bounded batches for exact payment.

16. Who may finalize refunds?

Anyone may call `enableRefunds` after the applicable deadline. That caller cannot take the refunds.

17. Who may redeem refunds?

Only the actual current owner of every supplied ticket ID.

18. Who receives a refund after a ticket transfer?

The owner who burns the unredeemed ticket chooses the USDC destination. Earlier owners have no stored refund credit.

19. Can a ticket refund be redeemed twice?

No. Successful redemption burns the ticket. A second `ownerOf` or burn attempt fails.

20. Does a refunded ticket still exist?

No. It is burned, so there is no post-refund souvenir token.

21. What happens if the threshold is missed?

With a valid callback, one ticket wins cash, the sponsor receives the post-fee remainder, the treasury receives the fee, and the recovery recipient gets the NFT.

22. Does cash fallback refund everyone?

No. It is a successful one-winner branch.

23. How is the protocol fee calculated?

`floor(grossSales x 500 / 10,000)`, which is 5% of gross after integer flooring. It applies only after a valid callback.

24. Is winner cash 80% of gross?

No. It is 80% of the post-fee distributable pot.

25. Who pays the randomness fee?

The account that calls `requestDraw` pays Pyth's current fee in Base-native currency. The USDC ticket pot does not reimburse it.

26. What happens to request overpayment?

It is returned immediately. If the requester cannot accept the return, the complete request reverts. There is no native pull claim.

27. Why do users have to claim assets?

Independent actions prevent a failing sponsor, treasury, winner, or prize destination from blocking settlement or other claimants.

28. Can anyone claim quote proceeds for me?

Anyone may call `claimQuoteFor(account)`, but payment can go only to that account.

29. Can another person redirect my quote claim?

Not through `claimQuoteFor`. Only the claimant can use `claimQuote` to choose another destination.

30. Can anyone claim my winning ticket for me?

No general claim-for function exists for bearer awards. The actual owner must redeem or transfer the ticket to a capable owner.

31. What happens if my claim fails?

The transaction reverts and should restore the ticket, liability, and claim state. The external cause may still prevent every later attempt.

32. Can a frozen token trap a claim?

Yes. Contract accounting cannot force USDC or a prize contract to transfer.

33. Can any ERC-20 be used?

No. One factory uses one immutable quote-token address. The reviewed deployment model expects official chain-specific USDC with exact, non-rebasing transfer behavior.

34. Can any NFT be raffled?

The factory requires a contract reporting ERC-721 support, but that is not a full safety review. Malicious, upgradeable, paused, or dishonest NFTs remain dangerous.

35. Can the contracts be upgraded?

No. Raffles are independent constructor deployments without proxy upgrade paths. The factory and Lens are also deployed as non-upgradeable contracts.

36. Is there an administrator rescue function?

No generic rescue exists. A narrow same-factory protocol-owned claim helper covers only specific registered-raffle cases and pays only the holder raffle's fixed recovery recipient.

37. What happens to unrelated NFTs sent to a raffle?

Unsafe transfers can force them in, but they are outside accounting and there is no administrator sweep. They may remain trapped.

38. Does the frontend control settlement?

No. It prepares transactions and displays data. Contracts and onchain state determine whether an action succeeds.

39. What if the subgraph disagrees with the contract?

Use the contract and a current authenticated read. The subgraph is non-authoritative.

40. Has raffle.fun been independently audited?

Independent external audit: Not completed. The available evidence is an internal adversarial review and testing campaign.

41. Is raffle.fun deployed?

Deployment status: Not deployed and not authorized for user funds. Testnet deployment status: No verified Base Sepolia deployment record.

42. Is raffle.fun legal everywhere?

No such claim is made. Legal treatment depends on the sponsor, prize, participants, jurisdiction, marketing, consideration, operating model, and other facts.

43. Are raffle tickets investments?

No. They are chance-based bearer entries and most win nothing. This paper does not describe them as investments or promise value.

44. What should I verify before interacting?

Verify the chain, deployment record, factory registration, contract source, exact raffle terms, prize and USDC contracts, audit status, legal status, wallet transaction, recipient capability, and your ability to bear a complete loss.

PART XIV

Conclusion

raffle.fun makes custody, bearer ownership, timing, randomness authentication, accounting, liveness failure, and claims explicit without an existing-raffle admin.

PART XIV

Conclusion

83. What raffle.fun Makes Possible

raffle.fun provides a concrete, inspectable mechanism for a sponsor to escrow one NFT, sell sequential bearer tickets in one factory-wide USDC, request one Pyth random value, and expose deterministic NFT, cash, refund, or empty-result asset paths.

The mechanism reduces dependence on a manual custodian or discretionary winner selector. It also makes tradeoffs visible: existing raffles cannot be upgraded or rescued, ticket rights can be trapped by bad transfers, external assets can freeze, and legal compliance remains outside the code.

84. Final Takeaways

1. Each raffle is an independent, non-upgradeable constructor deployment holding one exact ERC-721 prize.
2. One factory uses one immutable USDC address; ticket price is the complete gross payment and the threshold is not a sales cap.
3. Tickets are transferable bearer rights in every status and burn to consume winning or refund value.
4. One authenticated Pyth Entropy request selects `(random % totalTickets) + 1`; fixed deadlines route liveness failure to fee-free refunds.
5. A successful cash fallback selects one winner. It does not refund everyone.
6. Sponsor and treasury proceeds are independent pull claims; bearer awards and refunds transfer on redemption.
7. Factory ownership affects future creation only and cannot patch, settle, seize, or rescue an existing raffle.
8. The protocol remains experimental, internally reviewed, independently unaudited, legally unreviewed, and not deployed or authorized for user funds.

TECHNICAL APPENDICES

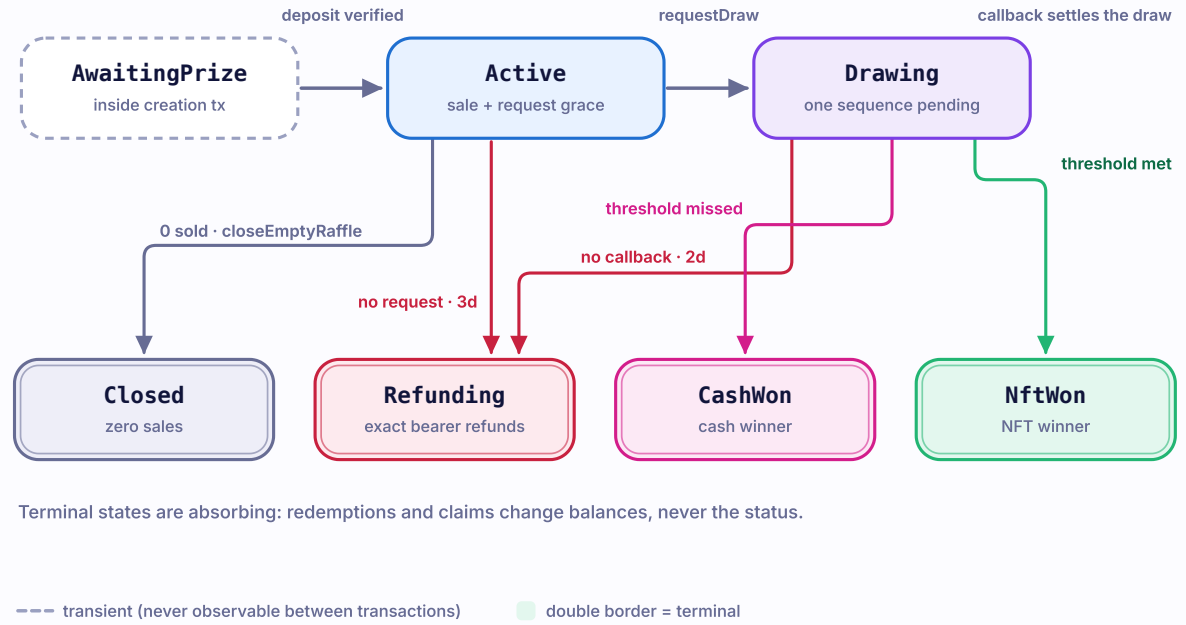
Exact Reference

The appendices summarize the state machine, formulas, contracts, events, invariants, supported assets, review evidence, deployment gates, legal risks, terms, and sources.

APPENDIX A

Exact State Machine

FIGURE 21 **Complete lifecycle**



Seven status values represent both lifecycle and economic result. Asset claims do not create additional status transitions.

FROM	TO	CALLER	EXACT PRECONDITIONS	CORE EFFECTS	EVENT	TICKET TRANSFER
construction	`AwaitingPrize`	factory through constructor	`msg.sender` equals declared factory; fixed parties and dependencies nonzero; unsafe fixed protocol destinations rejected	store all immutables and metadata	none	no tickets exist
`AwaitingPrize`	`Active`	factory-operated prize safe transfer	exact prize contract and ID; sponsor is `from`; factory is operator	activate raffle	`PrizeDeposited`	no tickets yet
`Active`	`Closed`	sponsor before end, anyone at/after end	`totalTickets == 0`	expose sponsor-side prize claim	`EmptyRaffleClosed`	no tickets exist
`Active`	`Drawing`	anyone	`timestamp >= endTime`; tickets exist; `timestamp < requestGraceDeadline`; sufficient current fee; Entropy request and immediate excess return succeed	store request time and one sequence	`DrawRequested`	remains allowed
`Active`	`Refunding`	anyone	tickets exist; `timestamp >= requestGraceDeadline`	move `unsettledPot` to `remainingRefundLiability`	`RefundsEnabled` with request false	remains allowed
`Drawing`	`NftWon`	authenticated Entropy callback	not in flight; exact sequence; threshold met	select ticket; clear unsettled pot; credit fee and sponsor	`RaffleResolved`	remains allowed until burn
`Drawing`	`CashWon`	authenticated Entropy callback	not in flight; exact sequence; threshold missed	select ticket; clear unsettled pot; record winner cash; credit fee and sponsor	`RaffleResolved`	remains allowed until burn

FROM	TO	CALLER	EXACT PRECONDITIONS	CORE EFFECTS	EVENT	TICKET TRANSFER
`Drawing`	`Refunding`	anyone	`timestamp >= callbackDeadline`	move unsettled pot to refund liability	`RefundsEnabled` with request true	remains allowed

Invalid, in-flight, stale, duplicate, or post-terminal callbacks emit `EntropyCallbackIgnored` without a transition. Claims and burns do not change the terminal status. They update ticket existence, prize-claimed state, and liabilities.

Status reference

STATUS	MEANING	CLAIMS AVAILABLE
`AwaitingPrize`	transient constructor state inside atomic factory transaction	none
`Active`	sale or post-sale request grace	purchases during window; request after end; empty close; failure finalization at grace
`Drawing`	one Entropy sequence pending	valid callback or timeout finalization
`NftWon`	threshold met after valid callback	winning bearer burns for NFT; sponsor and treasury pull USDC
`CashWon`	threshold missed after valid callback	winning bearer burns for cash; sponsor and treasury pull USDC; recovery recipient claims NFT
`Refunding`	liveness failure finalized	current bearers burn for refunds; recovery recipient claims NFT
`Closed`	zero-sales close	recovery recipient claims NFT

APPENDIX B

Exact Formulas

Let:

- `P` = raw ticket price;
- `Q` = purchase quantity;
- `T` = total tickets ever minted;
- `G` = gross sales;
- `F` = protocol fee;
- `D` = post-fee distributable pot;
- `W` = cash winner amount;
- `S` = sponsor amount;
- `R` = remaining refund liability;
- `C` = aggregate sponsor and treasury quote claims.

Purchases and sales

```
grossPurchase = P x Q
```

```
G = P x T
```

Tickets from one purchase are:

```
firstTicketId = previousTotalTickets + 1
```

```
lastTicketId = previousTotalTickets + Q
```

Successful resolution

```
F = floor(G x 500 / 10,000)
```

`D = G − F`

When threshold is met:

`S = D`

`W = 0`

When threshold is missed:

`W = floor(D x 8000 / 10,000)`

`S = D − W`

Therefore `F + W + S = G` in cash outcome and `F + S = G` in NFT outcome.

Failed draw

At failure finalization:

`R = unsettledPot = G`

For a successful batch of `N` refundable tickets:

`refund = P x N`

`R_after = R_before − refund`

Aggregate quote accounting

`accountedQuoteBalance = unsettledPot + R + winnerCashLiability + C`

For a supported quote token:

`actualQuoteBalance >= accountedQuoteBalance`

The difference is unaccounted surplus and creates no claim.

Winner and odds

`winningTicketId = (uint256(randomNumber) % T) + 1`

`current holder share = tickets currently owned / T`

Pixel Passport raw-unit check

VALUE	HUMAN USDC	RAW UNITS
Ticket price	10.00	`10000000`
120-ticket gross	1,200.00	`1200000000`
120-ticket fee	60.00	`60000000`
120-ticket sponsor	1,140.00	`1140000000`
80-ticket gross	800.00	`800000000`
80-ticket fee	40.00	`40000000`
80-ticket cash winner	608.00	`608000000`
80-ticket sponsor	152.00	`152000000`
80-ticket failed-draw refunds	800.00	`800000000`

APPENDIX C

Contract Reference

RaffleFactory

Purpose. Validate and atomically create canonical raffles.

Immutable dependencies. Factory-wide quote token, Pyth Entropy v2, callback gas limit.

Mutable state. Future treasury, future creation pause, owner/pending owner, raffle count, registry mappings.

Public actions. `createRaffle`, `setProtocolTreasury`, `setCreationPaused`, `transferOwnership`, `acceptOwnership`, `renounceOwnership`, and reads.

Access. Creation is open while unpaused. Policy writes are owner-only.

External calls. Raffle constructor deployment, prize ERC-165, safe transfer, `ownerOf`, and raffle `status` verification.

Security notes. Nonreentrant atomic creation; future-only administration; runtime size margin is narrow.

Raffle

Purpose. Hold one NFT, mint transferable tickets, request randomness, account for one result, and expose claims.

Immutable dependencies and configuration. Factory, sponsor, recovery recipient, treasury, quote token, Entropy, prize contract and ID, raffle ID, price, threshold, start, end, callback gas.

Mutable state. Tickets, status, sales and liabilities, request sequence/time, winner, prize claim flag, metadata, quote claims.

Public actions. Purchase, zero-sales close, fee read, draw request, failure finalization, winner/refund redemption, quote claims, sponsor prize claim, selective protocol-owned recovery, ERC-721 transfers and approvals, and reads.

Access. No administrator. Rights are status-based, permissionless, actual-ticket- owner, fixed-recipient, or quote-claimant checks.

External calls. USDC balance/transfer, Entropy fee/request, prize safe transfer, ticket receiver callbacks, factory registry reads.

Security notes. Reentrancy guard, exact token deltas, storage-only callback, bounded loops, known protocol-destination checks, no rescue.

RaffleLens

Purpose. Aggregate wallet-oriented reads.

Immutable dependency. One canonical factory.

Storage responsibilities. None beyond immutable factory.

Public actions. Single and up-to-64 raffle views.

Access. Public reads, but only for factory-registered raffles.

External calls. Factory registry, raffle views, ticket ownership, dynamic Entropy fee through the raffle.

Security notes. Read-only; fee failure isolated; results can become stale after the queried block.

APPENDIX D

Event Guide

Indexers should also process inherited ERC-721 `Transfer`, `Approval`, and `ApprovalForAll` events.

EVENT	MEANING	IMPORTANT RECONSTRUCTION RULE
`RaffleCreated`	factory assigned and configured a new address	reverted transaction means no creation; later `PrizeDeposited` activates
`PrizeDeposited`	exact configured NFT entered escrow	status becomes `Active`
`TicketsPurchased`	exact USDC received and ticket range minted	combine with ERC-721 mint transfers; gross is event raw amount
ERC-721 `Transfer`	ticket minted, transferred, or burned	zero `from` is mint; zero `to` is burn; current owner is bearer
`EmptyRaffleClosed`	zero-sales raffle entered `Closed`	no request or quote liability
`DrawRequested`	one Entropy sequence accepted	record fee, excess returned, request time, callback deadline
`EntropyCallbackIgnored`	callback could not settle	do not create a terminal result
`RaffleResolved`	valid callback chose ticket and NFT/cash branch	record exact fee, winner cash, sponsor cash, and one terminal status
`RefundsEnabled`	request or callback liveness failure entered `Refunding`	event identifies whether request had been accepted
`WinningTicketRedeemed`	selected ticket burned for NFT or cash	update burn, cash liability, and prize claim when applicable
`RefundTicketsRedeemed`	owner burned a bounded batch for exact refund	decrease remaining liability by event amount
`QuoteClaimed`	sponsor or treasury claim transferred	decrease claimant and aggregate pull liabilities
`SponsorPrizeClaimed`	fixed recovery recipient withdrew NFT	set prize claimed
`ProtocolTreasuryUpdated`	future factory treasury changed	do not rewrite existing raffle treasury
`CreationPauseUpdated`	future creation pause changed	do not alter existing raffle status
Ownable events	factory ownership transition	future-policy authority only

APPENDIX E

Security Invariants

The internal catalog maps 110 practical invariants to unit, adversarial, fuzz, stateful, independent-fuzzer, symbolic, fork, static, and integration evidence. Core invariants are summarized below.

PLAIN-ENGLISH INVARIANT	TECHNICAL FORM OR EVIDENCE TARGET
Creation is all or nothing	registered implies exact prize owner and `Active`; revert rolls back ID and deployment
Only the factory constructs a raffle	constructor requires `msg.sender == params.factory`
Existing rules do not change	constructor immutables; no raffle owner or upgrade selector
Sales start inclusively and end exclusively	`timestamp >= startTime && timestamp < endTime`
Every paid ticket has one sequential ID	`totalTickets == grossSales / ticketPrice`; IDs 1 through total
Exact payment precedes mint	raffle balance delta equals `ticketPrice x quantity`
Tickets are bearer rights	actual `ownerOf` required at redemption; transfer stays open until burn
Known protocol destinations cannot trap tickets	transfer rejects self, factory, dependencies, prize, registered raffles
One request and one terminal result	monotonic status and stored sequence; no second request path
Callback cannot settle in flight	private guard remains true until sequence is stored
Wrong or duplicate callback is harmless	status/sequence/guard mismatch emits ignored event
Winner is always sold	`1 <= winningTicketId <= totalTickets`
Both oracle-liveness failures recover	exact grace and timeout boundaries reach `Refunding`
Callback and timeout are mutually terminal	first status change prevents the other result

PLAIN-ENGLISH INVARIANT	TECHNICAL FORM OR EVIDENCE TARGET
Successful branches charge the compiled fee	`fee = floor(gross x 500 / 10,000)`
Failed branches charge no fee	`enableRefunds` creates no sponsor or treasury claim
Threshold equality awards NFT	`totalTickets >= minimumTickets` selects `NftWon`
Cash branch allocates every unit	fee + winner + sponsor = gross
Every refund ticket pays once	successful burn reduces liability by exactly one ticket price
Outgoing token failure preserves rights	exact debit/credit check reverts complete transaction
Prize leaves at most once	burn or fixed claim plus `prizeClaimed`; retry on revert
Callback work is bounded	no token/user call and no sold-ticket loop
Admin affects future only	only treasury/pause and ownership writes on factory
Onchain state is authoritative	Lens authenticates registry; SDK/subgraph/frontend have no settlement selector

The complete catalog is in `docs/SECURITY-INVARIANTS.md`. Evidence mapping is not formal verification.

APPENDIX F

Supported Asset Model

Supported prize assumptions

- deployed contract reporting ERC-721 through ERC-165;
- truthful and stable `ownerOf` behavior;
- available safe transfers and receiver calls;
- no malicious burn, reentry, or upgrade that violates custody;
- metadata and rights independently evaluated.

Supported quote assumptions

- the factory's chain-specific USDC contract is correct;
- balances are truthful;
- transfers are exact, non-rebasing, and continuously available;
- no fee, tax, sender surcharge, receiver haircut, or unexpected rebase;
- issuer pause, blacklist, and upgrade powers are understood.

Chain and oracle assumptions

- Base executes EVM semantics and includes transactions;
- timestamps and ordering remain within the chain model;
- the configured Pyth address is authentic;
- the random value is correct under Pyth's external security model;
- fees and callback-gas policy remain operable.

Explicit exclusions

Issuer freezes, arbitrary malicious token code, dishonest ownership or balance reads, burned escrowed NFTs, lost keys, arbitrary incapable ticket destinations, unrelated forced NFTs, forced native value, direct donation recovery, universal censorship, and chain halt are outside the supported recovery claim.

APPENDIX G

Audit and Test Evidence

Document status: Security review status: Internal adversarial review only. Independent external audit: Not completed.

The internal audit used source identity `a2120f5e163dc3641d9864773febbfedca047edb` plus dirty-worktree fingerprint `55589ce1fce0bd5579e60df747796575df9b0d96`. The final reviewed implementation is committed at `f165e4c1d8f5d093fe0a36094f79a29857c26286`.

Findings: 0 Critical; 4 High fixed; 1 Medium fixed; 2 Low fixed; 1 Informational accepted. No unresolved Critical, High, or Medium supported-asset finding remained in the internally reviewed source. This does not imply no unknown vulnerability exists.

Key completed evidence includes 70 default Foundry passes, 660,000 critical fuzz cases, 10,000 differential sequences, 10,240,000 strict release handler calls, 200,421 Echidna transactions, 113,261 Medusa calls, a 52-mutant 100% killed sample, five Halmos checks, production coverage of 100% lines/functions and 98.84% branches, static analysis, gas and size checks, two pinned Base fork tests, and SDK/subgraph/web gates.

Blocked or incomplete evidence: Mythril not executed due Python compatibility; SMTChecker inconclusive; Certora not configured; no whole-protocol formal proof; no independent external audit; no monitored testnet campaign.

APPENDIX H

Deployment Verification Checklist

No item below should be inferred complete merely because local tests pass.

- [ ] exact final source commit and dependency lock selected;
- [ ] all repository and security gates rerun from a clean checkout;
- [ ] official Base chain ID and current Pyth Entropy address verified from primary sources on release day;
- [ ] official Base USDC address, decimals, runtime code, proxy and issuer controls verified on release day;
- [ ] exact factory and Lens bytecode compiled with pinned settings;
- [ ] callback gas remeasured against exact deployment bytecode and provider policy;
- [ ] treasury Safe and factory-owner Safe reviewed;
- [ ] ownership acceptance completed and verified;
- [ ] factory quote token, Entropy, callback gas, treasury, owner, pending owner, and Lens binding read onchain;
- [ ] verified source reconciled byte-for-byte with deployment;
- [ ] signed deployment record written only after live validation;
- [ ] frontend and subgraph configured to the exact chain and addresses;
- [ ] Base Sepolia monitored through NFT, cash, empty, both failure, partial refund, complete refund, recovery-helper, and retry branches;
- [ ] independent external audit completed for exact source and locks;
- [ ] Critical, High, and supported-asset Medium external findings remediated and re-reviewed;
- [ ] legal, sanctions, tax, consumer, promotion, gaming, licensing, and age review completed for the operating model;
- [ ] monitoring, incident response, private disclosure, and communication runbooks staffed.

APPENDIX I

Legal and Operational Risks

raffle.fun is experimental software. This paper explains a code state; it is not a promise, offer, authorization to deploy, or assurance of future performance.

Chance-based prize systems may be regulated differently across jurisdictions. Laws concerning gambling, lotteries, sweepstakes, contests, consumer protection, advertising, sanctions, anti-money laundering, tax, licensing, privacy, intellectual property, and age restrictions may apply. Treatment depends on the sponsor, prize, participants, jurisdiction, payment model, marketing, and operational facts.

The protocol does not itself guarantee legal compliance. Sponsors, operators, and users must obtain jurisdiction-specific legal and tax advice. This document is not legal, tax, investment, or financial advice. Ticket NFTs are chance-based entries and should not be described as investments.

NFTs and USDC may lose value, become unavailable, be frozen, or fail technically. Internal testing does not eliminate smart-contract risk. No one should deposit a valuable asset solely because of this paper.

Operational risks include key loss, malicious signers, wrong-network deployment, incorrect official addresses, stale frontends, compromised DNS, RPC censorship, missing monitoring, slow incident communication, and inability to patch existing raffles.

APPENDIX J

Glossary

Accounted quote balance: Sum of the four USDC liability categories recorded by a raffle.

Active: Status covering the sale and post-sale request grace before a request.

Base: Ethereum-compatible layer-2 network targeted by deployment tooling.

Bearer right: A right controlled by current ownership of an unburned ticket.

Basis point: 0.01 percentage point.

Burn: Destroy an ERC-721 token and clear its ownership record.

Callback: Separate Entropy transaction delivering randomness.

Cash fallback: Successful below-threshold result with one cash winner and no general refunds.

Claim: Authorized action that moves a recorded asset or liability.

Closed: Zero-sales terminal status.

Constructor deployment: Independent contract created with configuration fixed at deployment, rather than a clone or proxy initializer.

Drawing: Status with one accepted Entropy sequence pending.

Entropy: Pyth service supplying the random callback value.

Escrow: Contract custody of the configured prize under fixed functions.

Exact transfer: Transfer whose measured sender debit and recipient credit equal the accounted amount.

Factory: Contract that validates, deploys, registers, deposits, and verifies.

Gas: Native-currency cost of EVM execution.

Gross sales: Ticket price times total tickets ever minted.

Immutable: Not changeable through the deployed contract's exposed controls.

Indexer: Offchain service reconstructing state and history from events.

Lens: Read-only contract aggregating factory-authenticated views.

Modulo bias: Tiny distribution difference created when a finite random domain does not divide evenly by ticket count.

NftWon: Successful threshold-met result.

Nonreentrant: Guarded against nested entry into protected functions.

Oracle: External data service delivering information to a contract.

Pull claim: Recorded balance withdrawn later by the claimant.

Quote token: Factory-wide USDC used for price and cash accounting.

Recovery recipient: Fixed sponsor-side account authorized to claim the prize in cash, refund, or empty outcomes.

Refunding: Liveness-failure status in which current bearers burn for exact refunds.

Safe transfer: ERC-721 transfer that calls a contract receiver hook.

Sequencer: Base actor that orders layer-2 transactions.

Sponsor: Prize depositor, parameter chooser, and successful sponsor-cash claimant.

Subgraph: Event-indexed offchain data model.

Threshold: Sold-ticket boundary between NFT and cash after successful randomness.

Transaction: Signed request for the chain to execute contract code.

USDC: The intended factory-wide quote token; issuer-controlled and externally dependent.

Wallet: Key-management and transaction-signing software.

APPENDIX K

References

Primary raffle.fun sources

- Reviewed repository and contract commit: `f165e4c1d8f5d093fe0a36094f79a29857c26286`.
- Production contracts: `packages/contracts/src/Raffle.sol`, `RaffleFactory.sol`, `RaffleLens.sol`, interfaces, and `RaffleConstants.sol`.
- Executable evidence: `packages/contracts/test/` and generated Hardhat artifacts.
- Internal review: `packages/contracts/audit/`.
- Security invariants: `docs/SECURITY-INVARIANTS.md`.
- Architecture, state, randomness, threat, and deployment notes under `docs/`.
- SDK: `packages/sdk/`; subgraph: `packages/subgraph/`; frontend: `apps/web/`.

External primary references

- Solidity documentation
- Solidity security considerations
- ERC-20 token standard
- ERC-165 interface detection
- ERC-721 non-fungible token standard
- OpenZeppelin Contracts 5.x
- OpenZeppelin ERC-721 API
- OpenZeppelin access-control API

- Pyth Entropy
- Pyth Entropy EVM guide
- Pyth request and callback variants
- Pyth custom callback gas
- Base protocol overview
- Base transaction troubleshooting

References explain standards and dependencies. They do not endorse raffle.fun or convert the internal review into an independent audit.